

Project Cassandra

for <Catch22>



Presented by
G4 | TEAM PYGMY <3

Meet Our Team

Leads



PRODUCT
OWNER

Isaiah



SCRUM
MASTER

Jerrick

Developers



DEVELOPER

Arin



DEVELOPER

Jamesz



DEVELOPER

Yan Ting



DEVELOPER

Joyce

Today

-
- | | |
|----|--------------------------------------|
| 01 | The Problem |
| 02 | Goals & Impact |
| 03 | Live Demo |
| 04 | System Architecture |
| 05 | Key Design Decisions |
| 06 | How We Got Here |
| 07 | Project Management |
| 08 | Constraints & What's Next |
-

01

The Problem

Our Sponsor: Catch22

A fintech startup that uses retail traders as market signals

DOMAIN

**Fixed Income,
Currencies &
Commodities**

Our data: XAU/USD (Gold)

BUSINESS NEEDS

**Consistent profit
across
campaigns**

& near real-time decisions

OUR FOCUS

**Improving
trading decisions**

powered by our Judge model

MEET SALLY

Let's follow one
trader through
what
Catch22 does
today.



C22 POSTS

A CHALLENGE

**Catch22 announces a trading campaign on Discord,
2–3 times a week.**



SALLY JOINS IN

She signs up
alongside
~500 other
traders.



SALLY TRADES BY C22'S RULES

She trades inside
C22's rulebook,
not the real
market.



MEANWHILE...

MEET ALEX



Sally's trades tell **C22** what someone like Alex is likely doing in the real market.

C22 CAN

TAKE THE OTHER SIDE

C22 can reverse
Alex's trade,
betting the other
way to make a
profit.



But not every trade **should** **be reversed**



Reverse the
right trades
→ **C22 profits.**



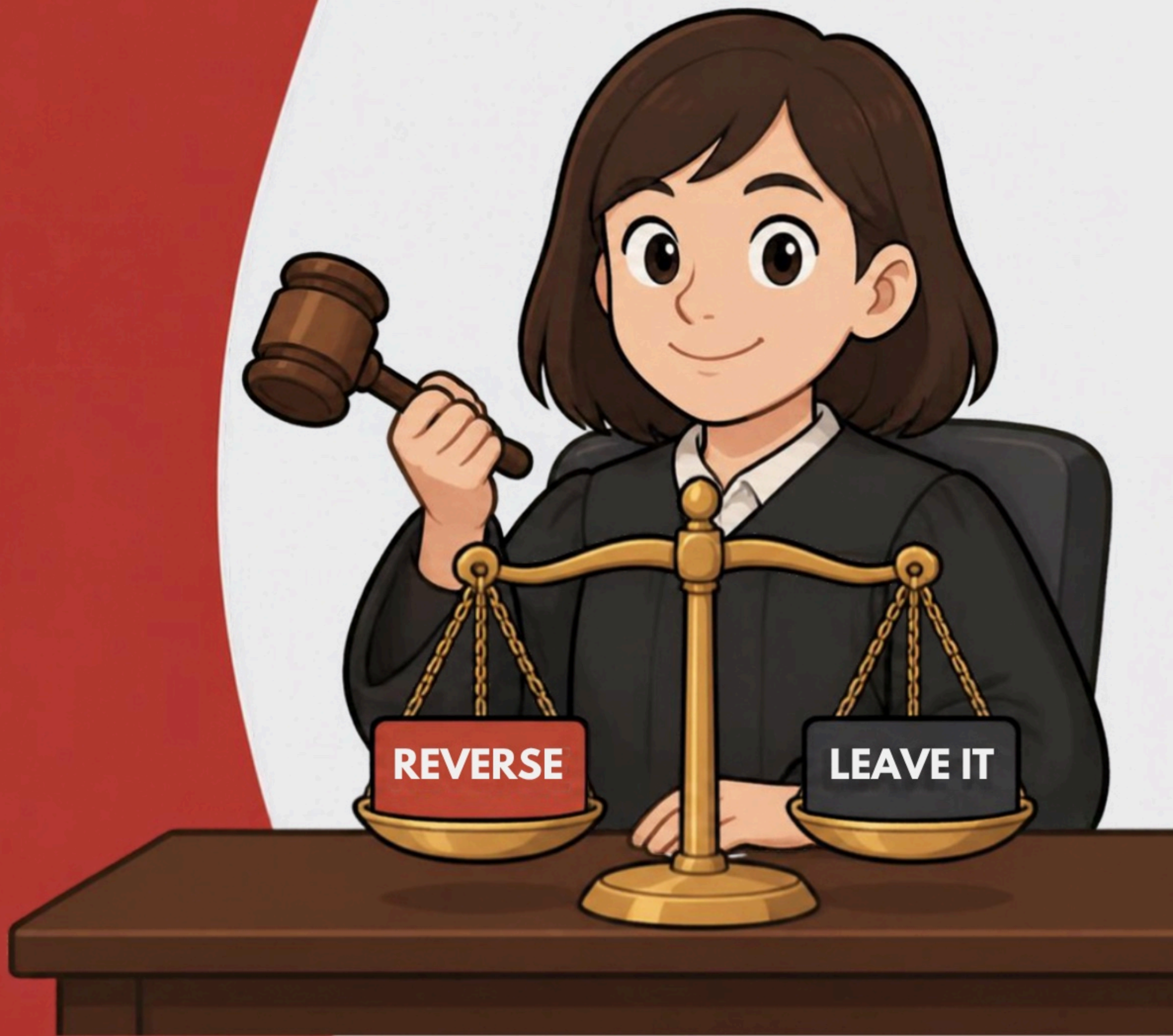
Reverse the
wrong ones
→ **C22 loses.**



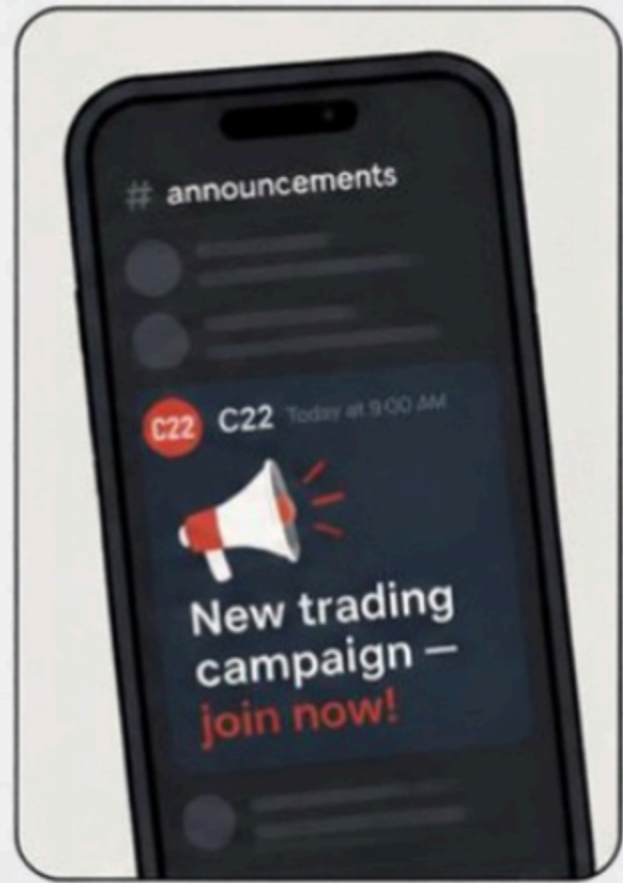
OUR FOCUS

THE JUDGE DECIDES,

trade by trade,
whether C22
should reverse
it.



THE WHOLE JOURNEY



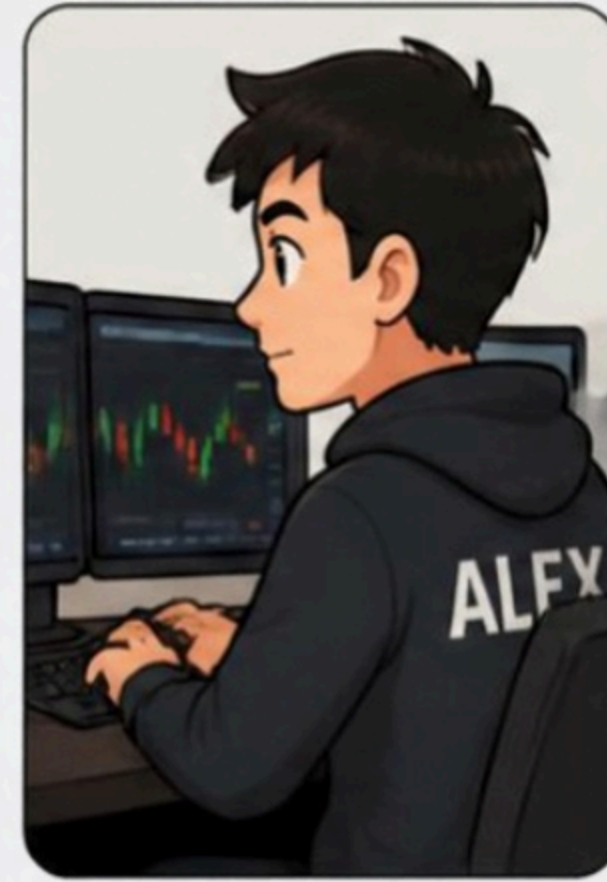
1. DISCORD
ANNOUNCEMENT



2. SALLY
JOINS



3. SALLY
TRADES



4. ALEX MIRRORS
IN THE REAL
MARKET



5. THE JUDGE
DECIDES

C22 can bet against Alex and **hopefully profit.**

The Judge decides when to.

Problem Space

Catch22's entire alpha depends on one critical decision:

“Which client’s trade do we fade (reverse) to maximise profit?”

Pain points

The existing system is **heavily constrained**

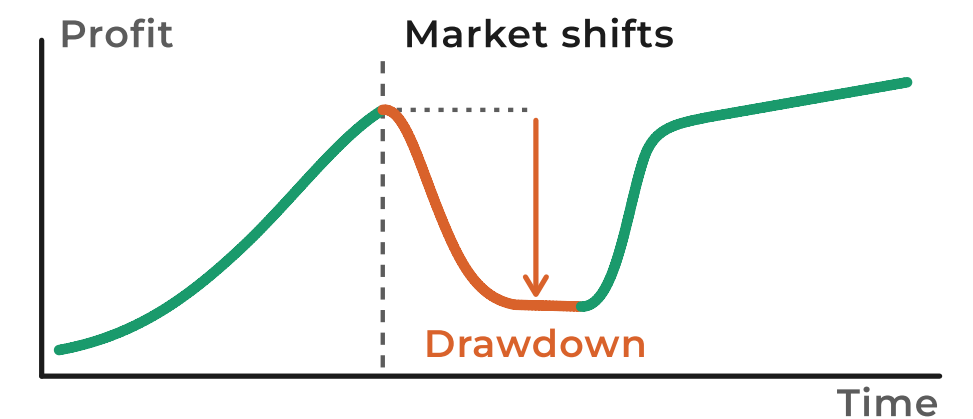
1 Static rules: a system that can't learn

Hard-coded rules go stale; they can't evolve with new trading patterns across campaigns.

2 Rules are fragile to market shifts

When conditions change suddenly, fixed rules can't adapt, and P&L swings with them.

Shape of today's P&L



02

Goals & Impact

The requirements we committed to, and
where we stand

Our Success Metrics

Key goals

To prove value, our model must clear all 3

Beat Fade Everything

Fading every single trade: the absolute floor

Beat Random

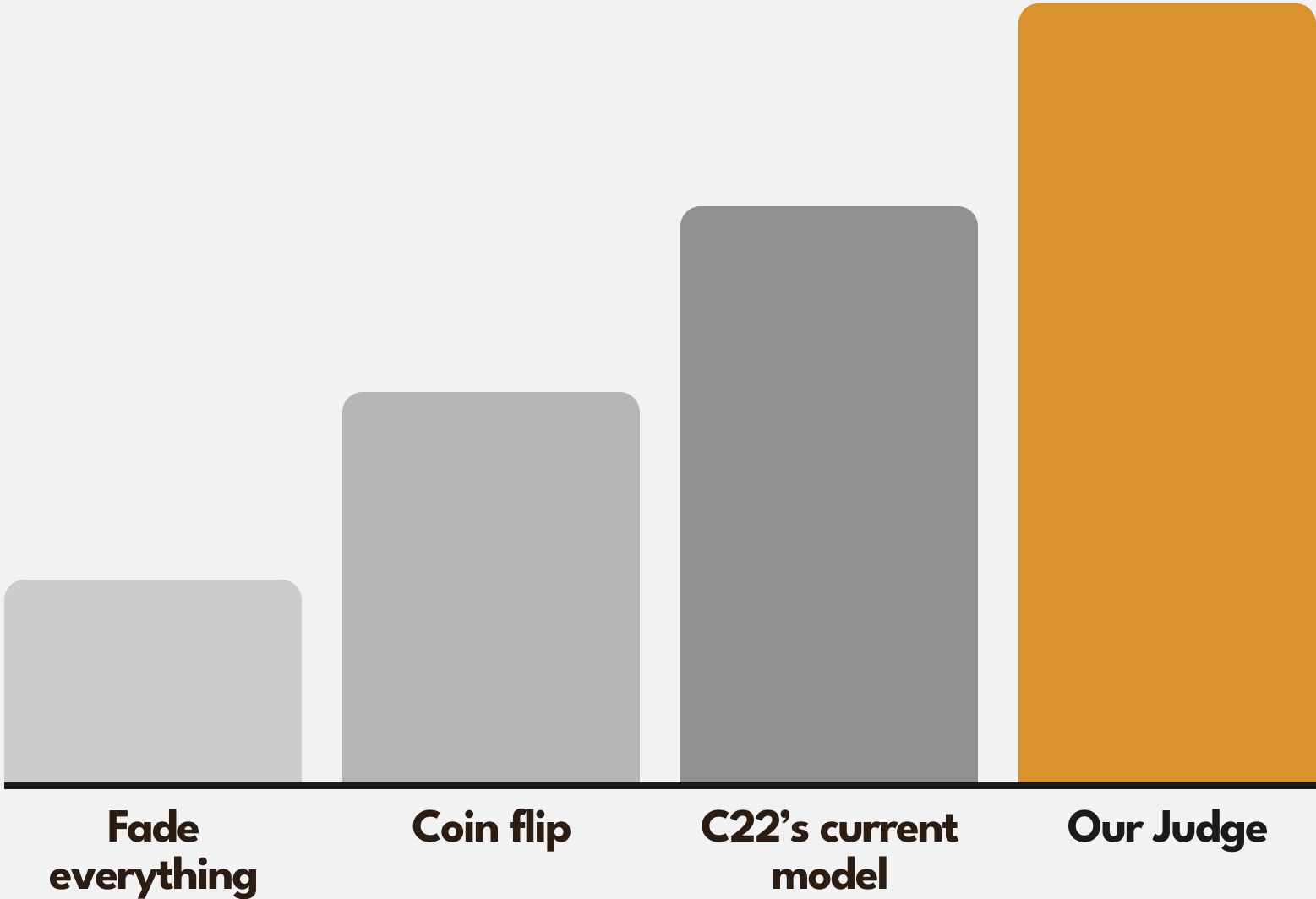
A fully randomised coin-flip baseline

Match C22's Current Rule

In-flight: on par with, or beating, the existing model

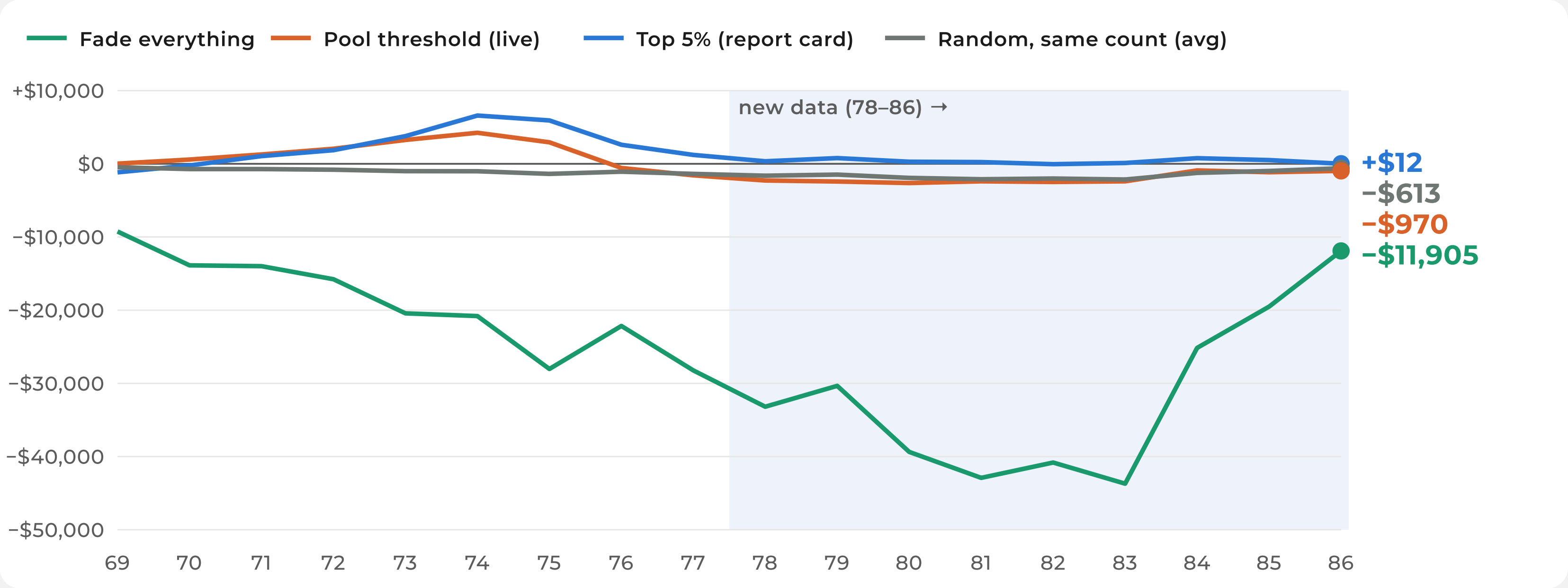
The staircase to climb

Goal: beat every benchmark



Illustrative target, not measured results.

We beat 2 of the 3 benchmarks



Fade everything ends at -\$11,905.
Our top 5% ends at +\$12, close to zero.

Beyond the Model

Closer to a research partnership

What C22 gets besides a better Judge

01

Fresh perspective

New eyes on feature engineering

02

New modelling

Approaches beyond the current rules

03

End-to-end suite

ETL, the Judge, monitoring and paper trading: a full platform, not just a judge

03

Live Demo

Watch the Judge decide in real time,
replayed at 15× speed

04

System Architecture

How a trade becomes a decision in real
time

Non-Functional Requirements

Data integrity & safety

Data reliability

 Trade records are stored regardless of operating conditions.

Auditability

 All execution context, judge decisions and errors are recorded.

Idempotency

 Duplicate requests are processed without double-counting outcomes.

Execution & performance

Processing latency

 Median time for price change to exceed acceptable threshold is 0.6-0.85s.

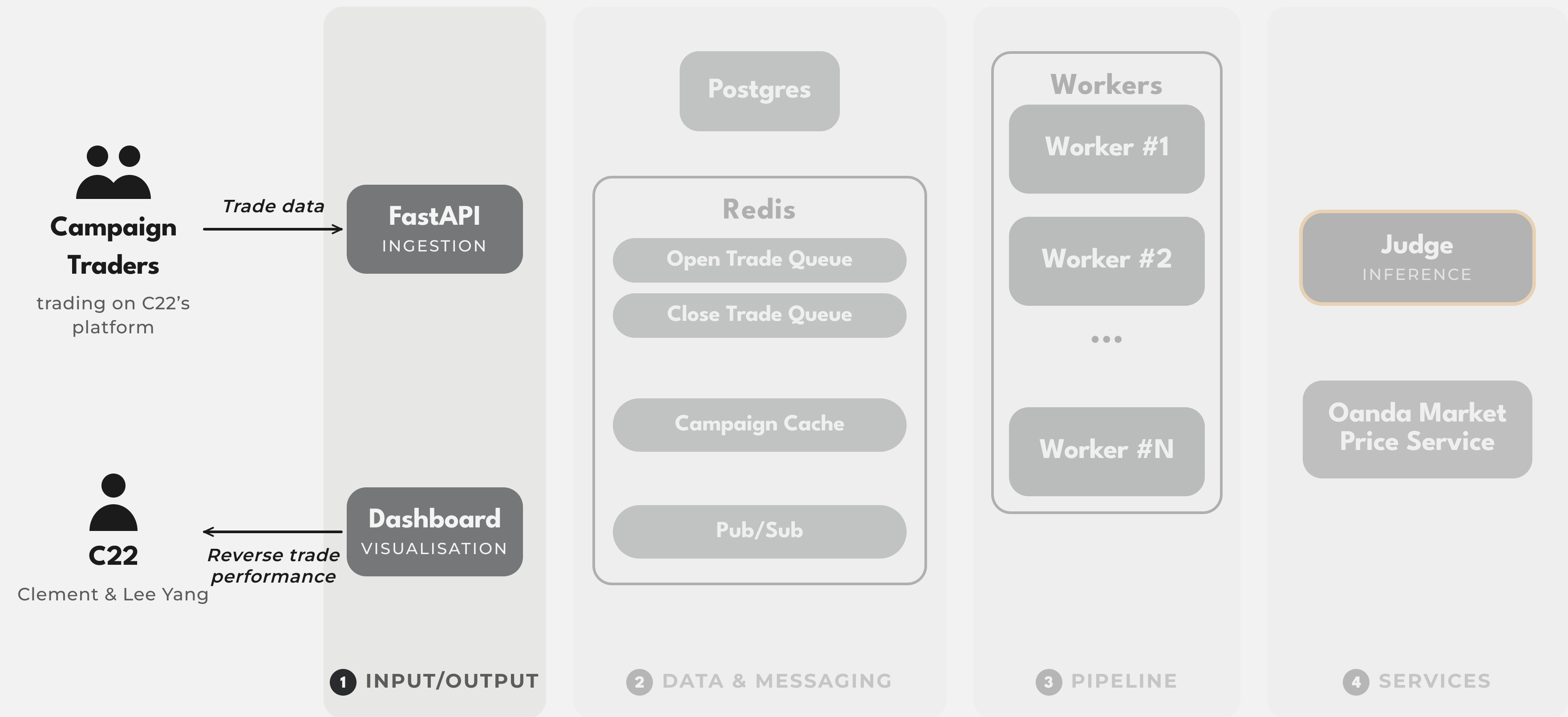
Concurrency & throughput

 Peak periods send up to 19 trades per minute and 5 per second based on past campaigns.

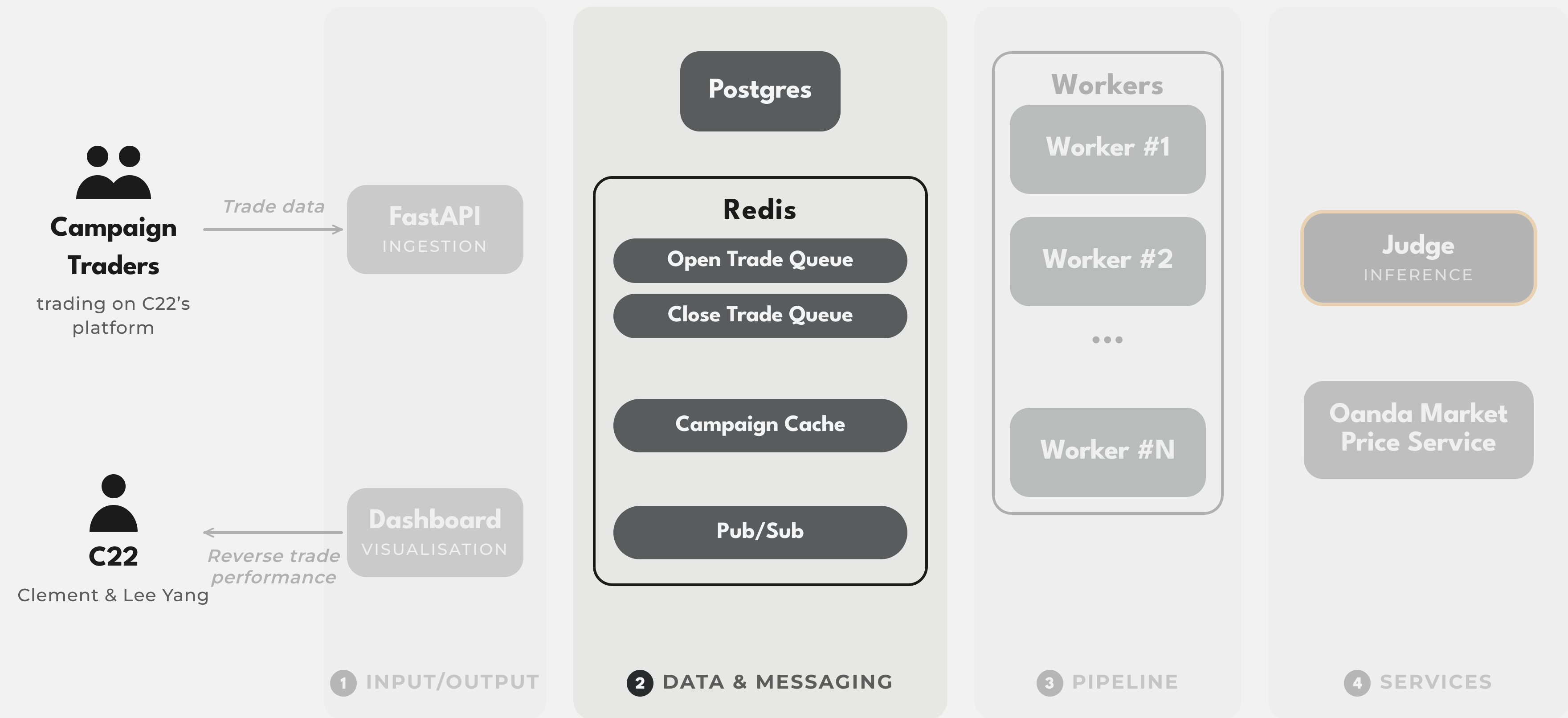
Sequencing

 Trades are processed sequentially (open comes before close).

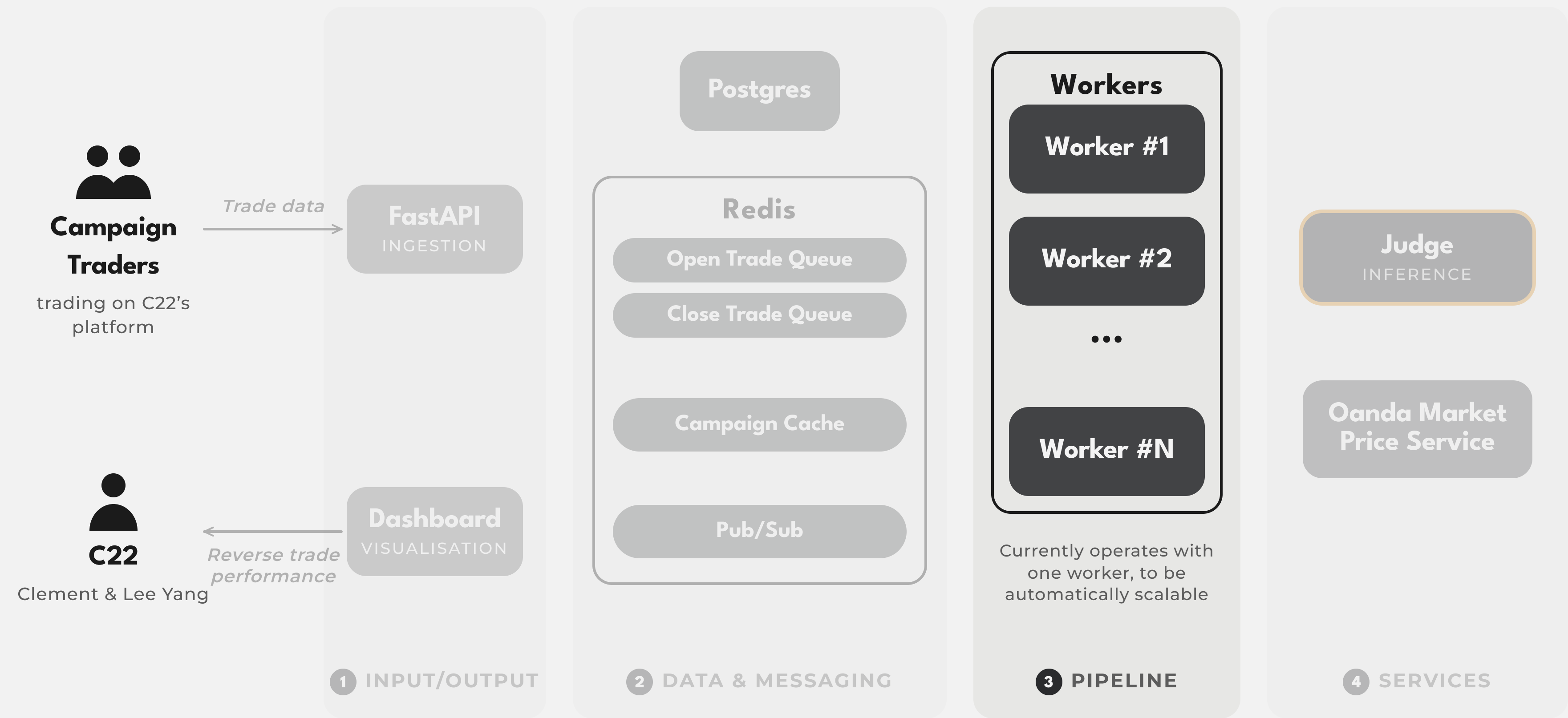
Project Cassandra Architecture



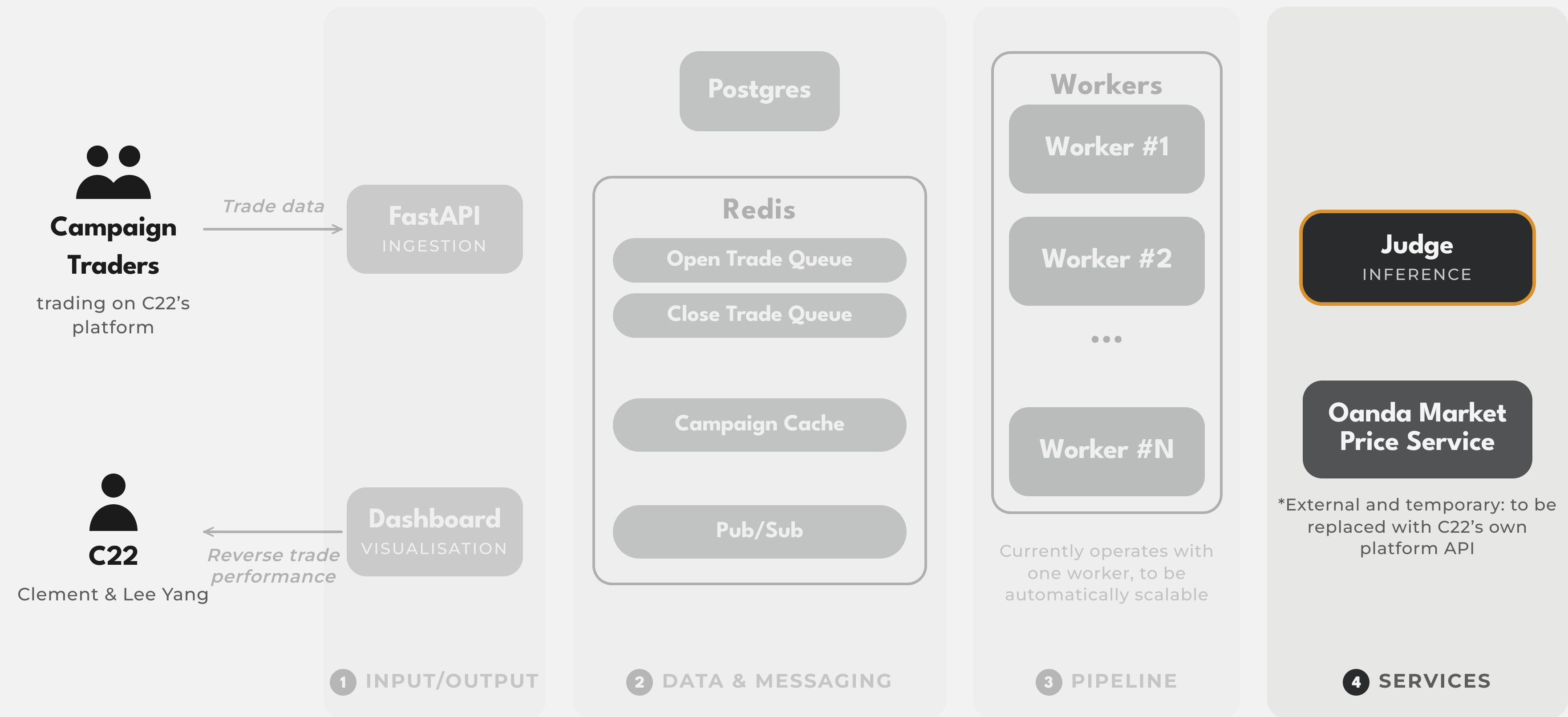
Project Cassandra Architecture



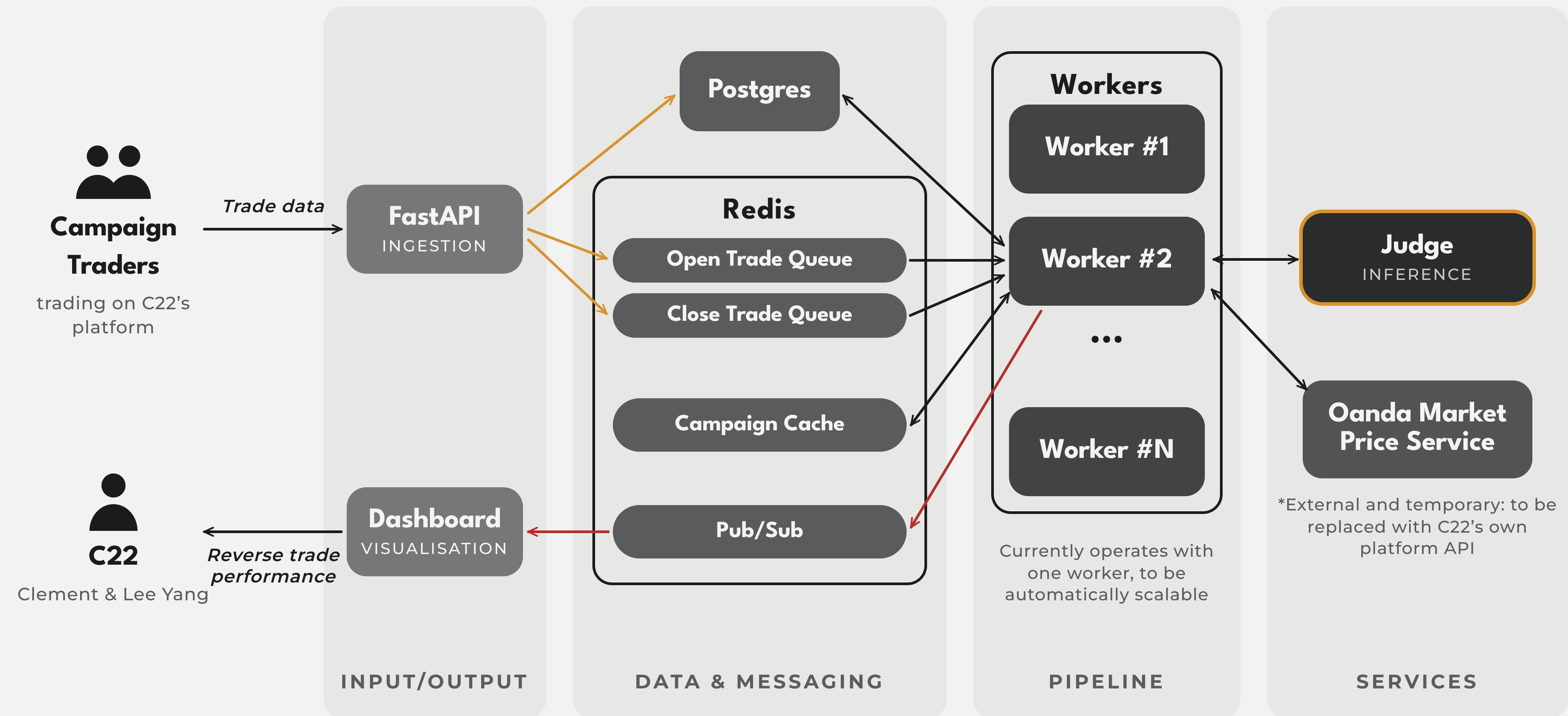
Project Cassandra Architecture



Project Cassandra Architecture

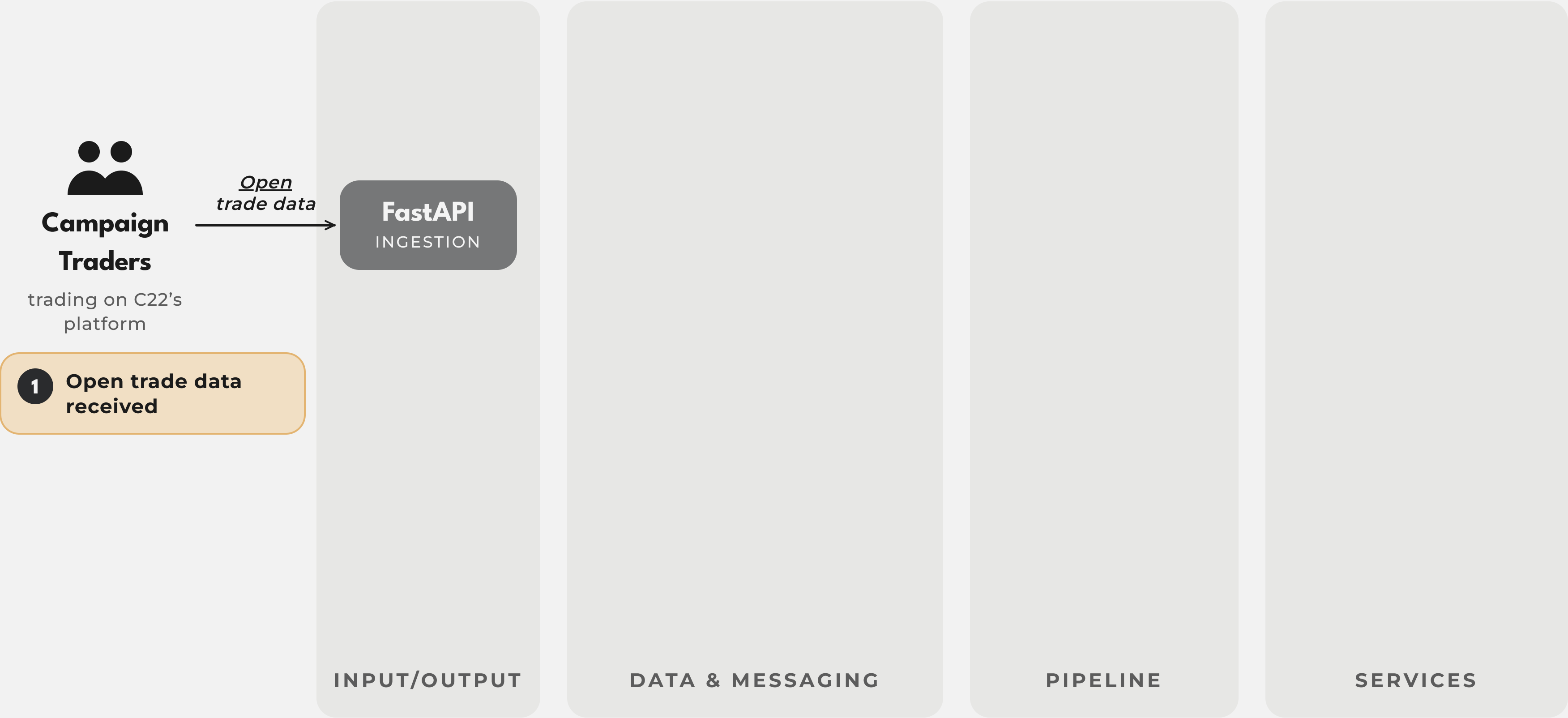


Project Cassandra Architecture

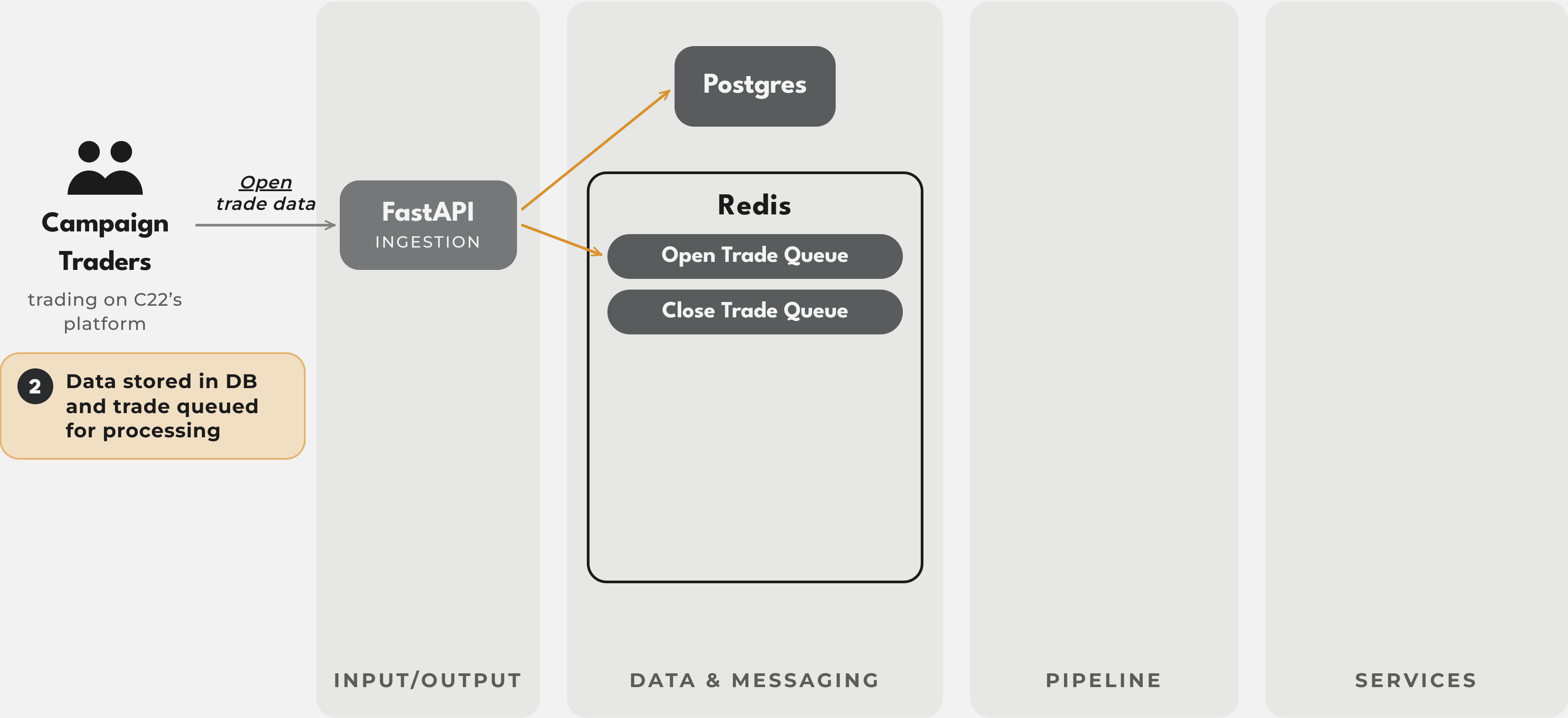


Open Trade Flow

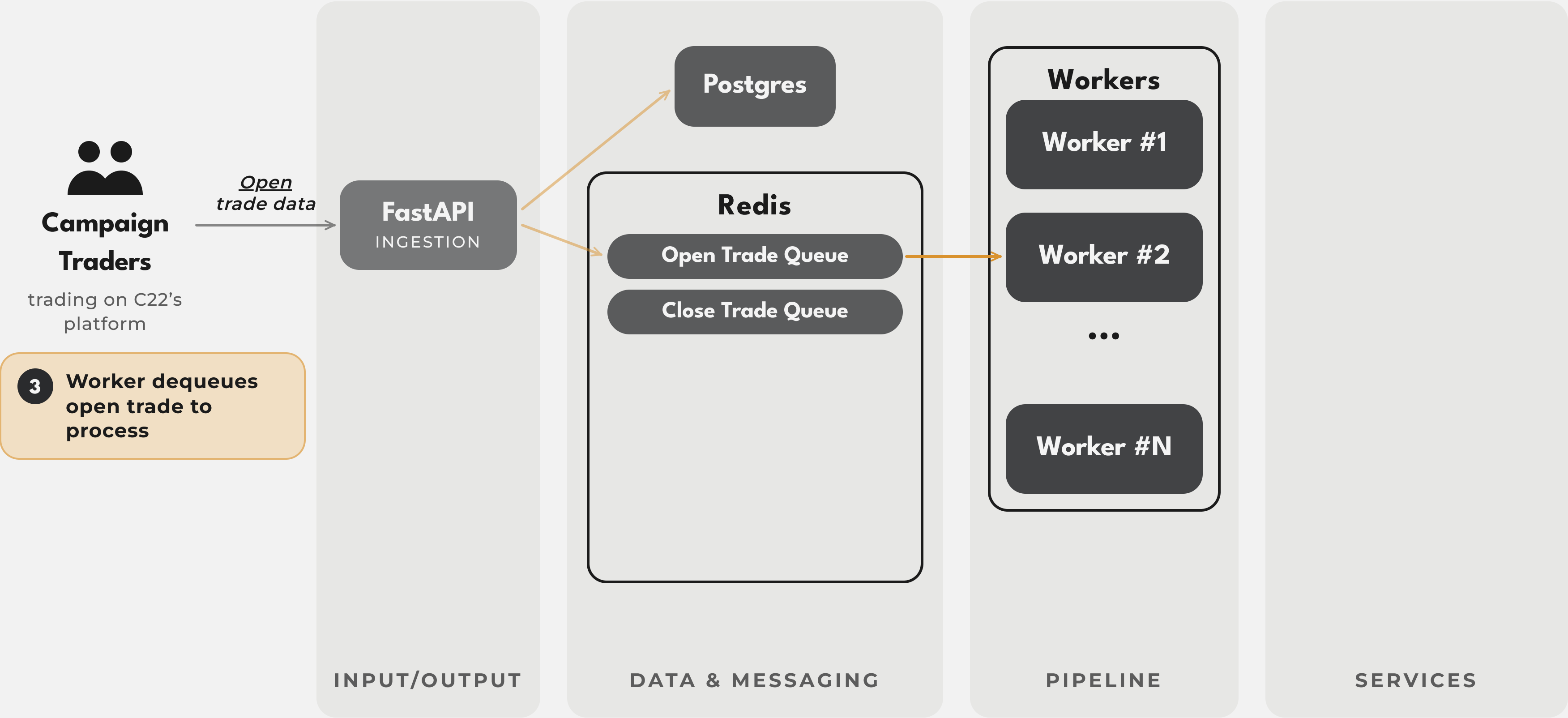
STEP 1 OF 6



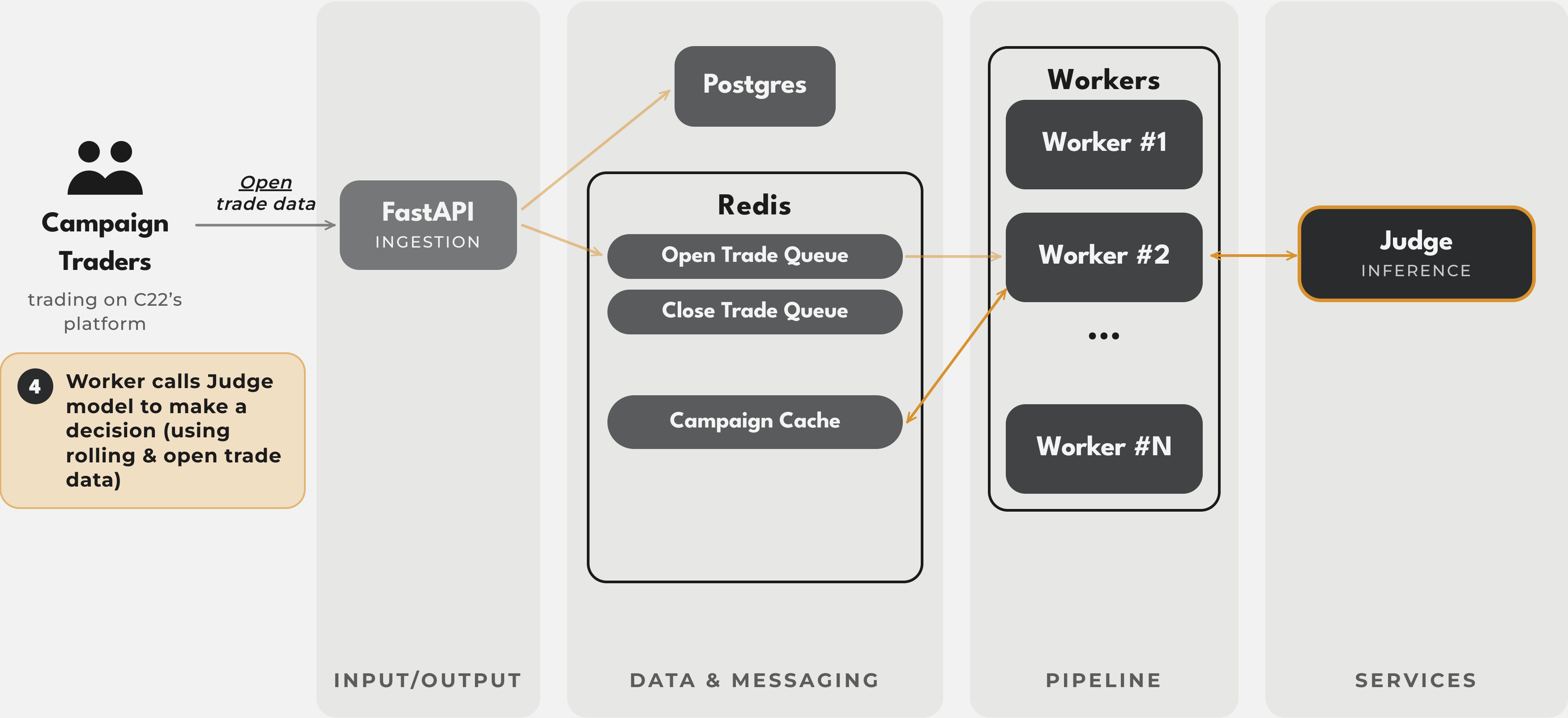
Open Trade Flow



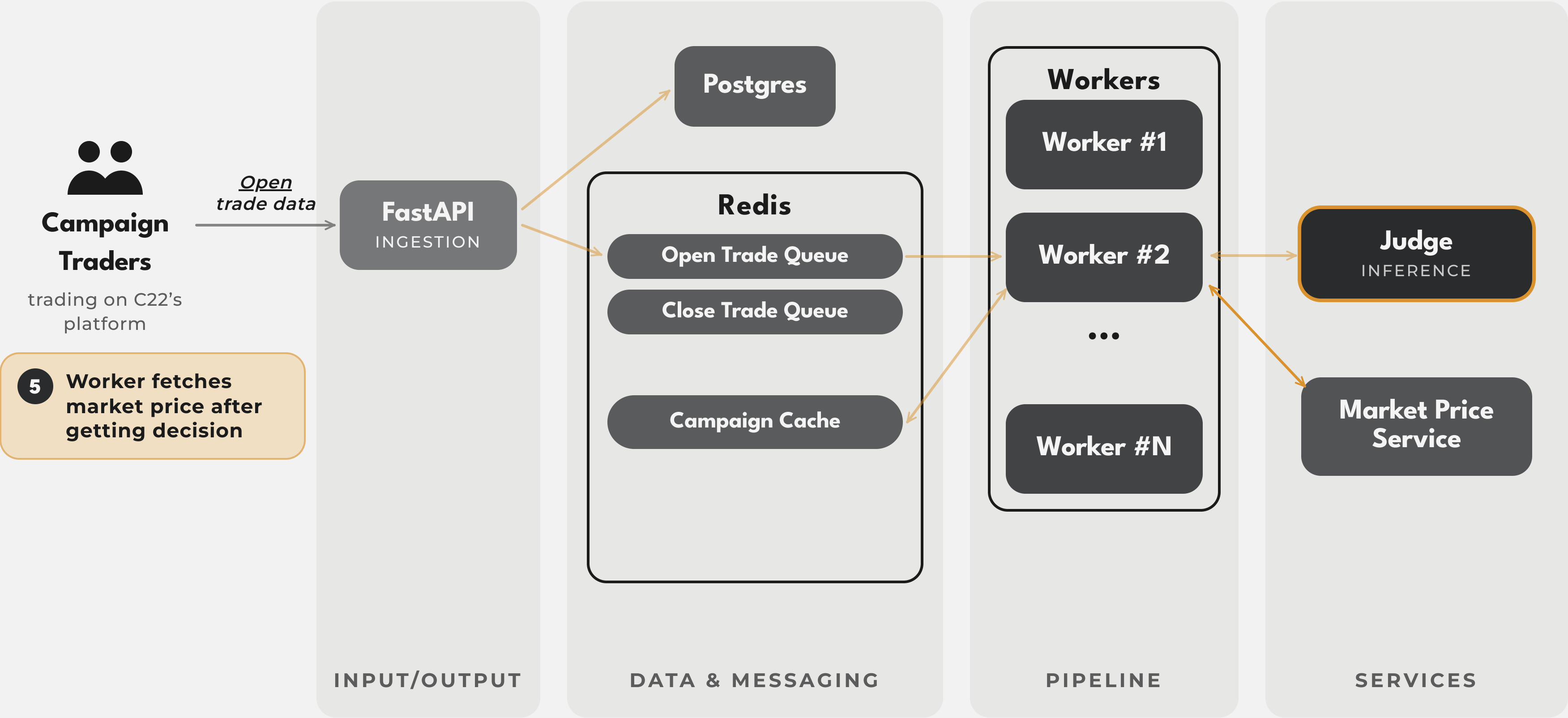
Open Trade Flow



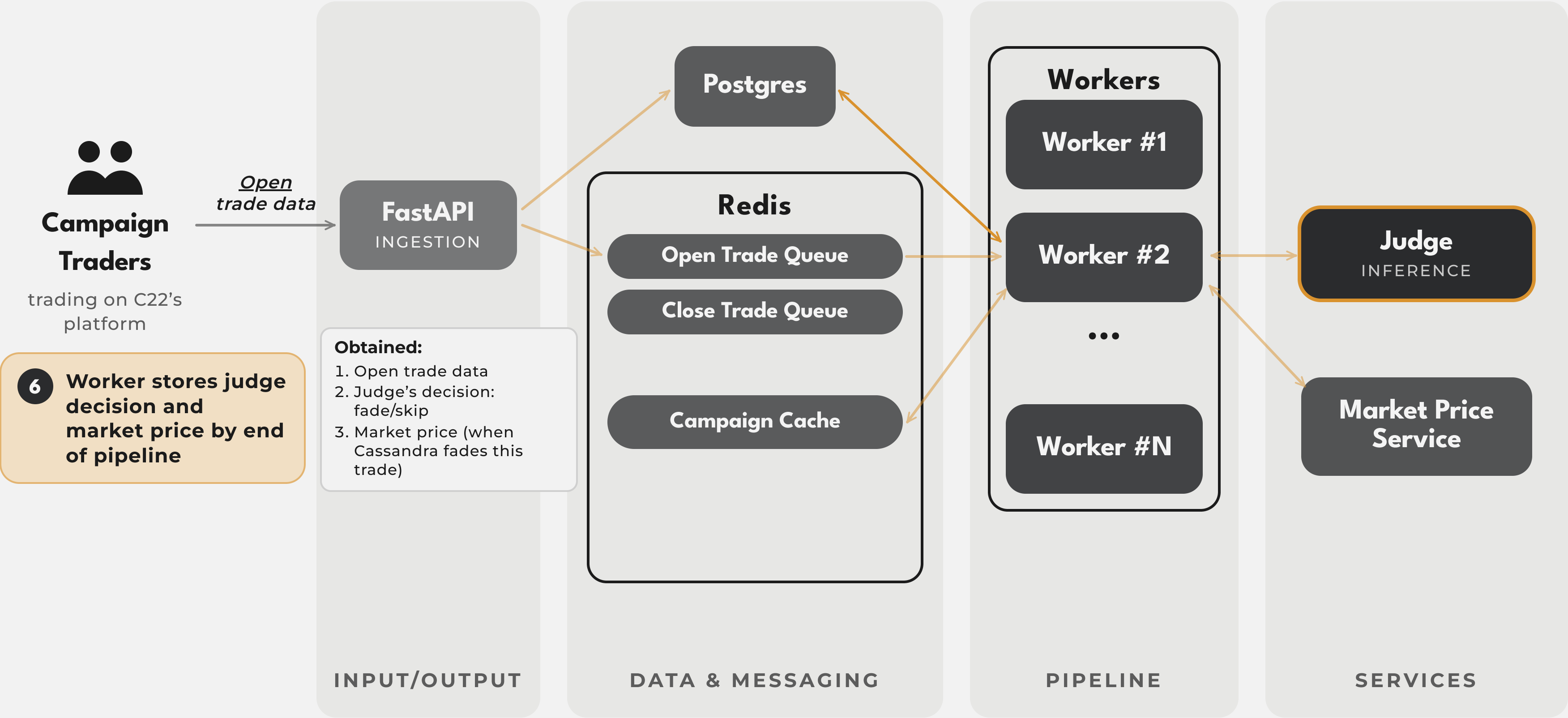
Open Trade Flow



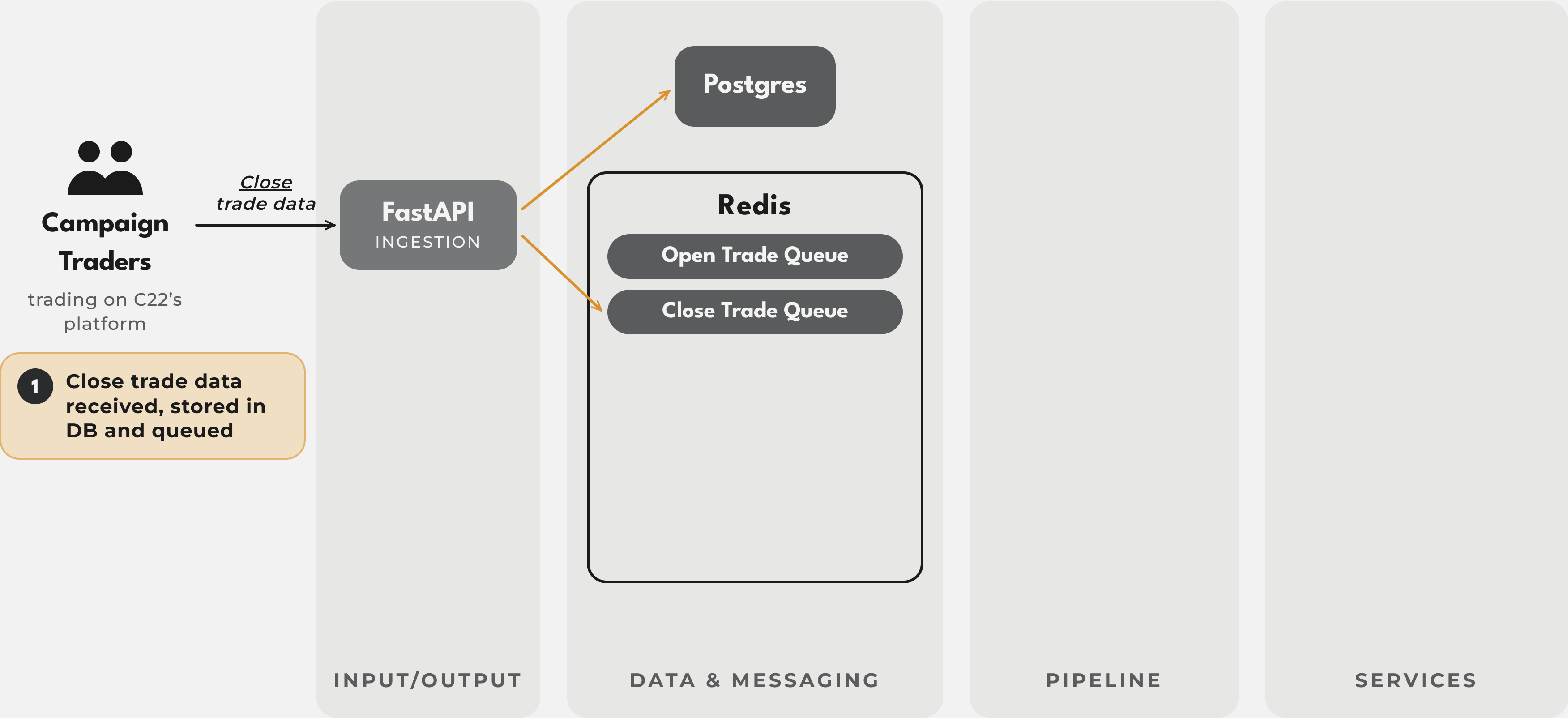
Open Trade Flow



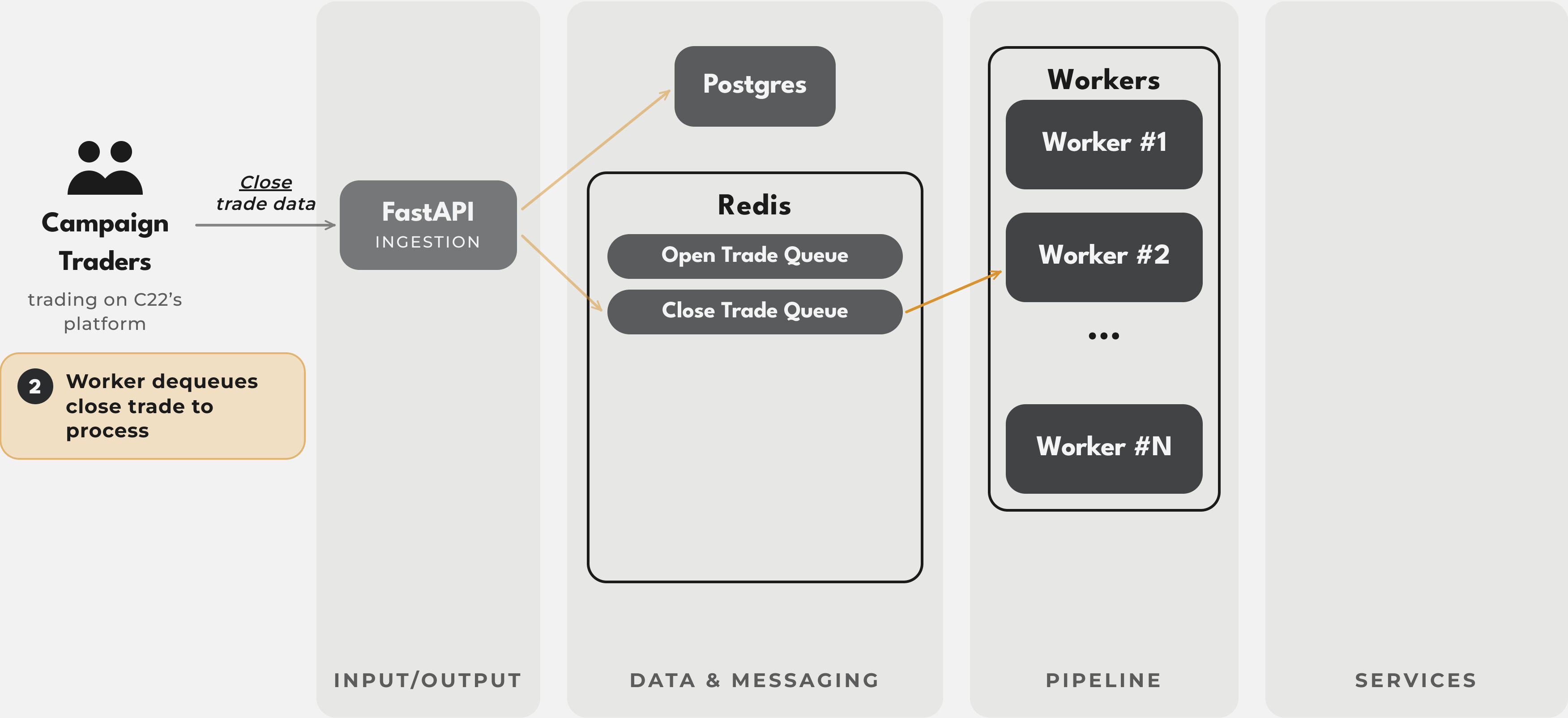
Open Trade Flow



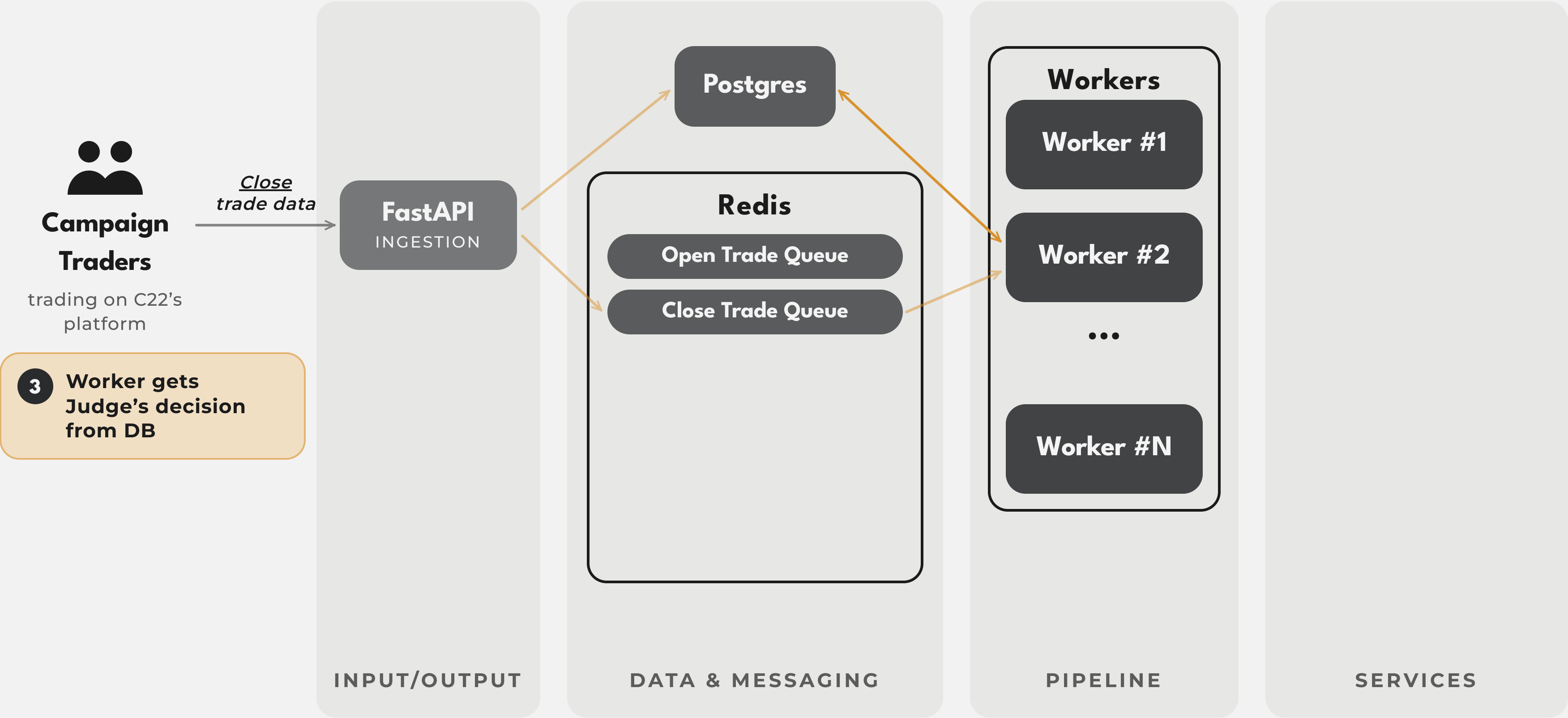
Close Trade Flow



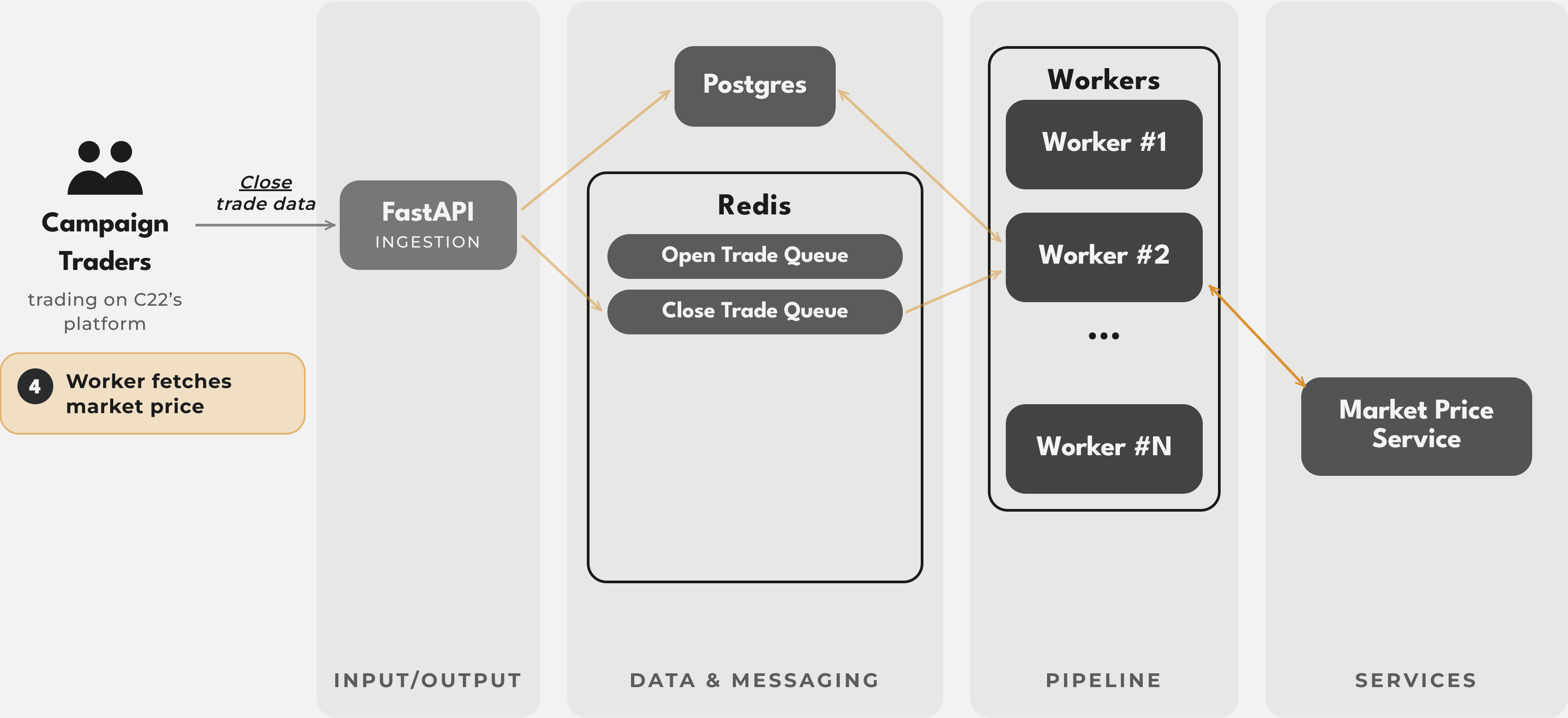
Close Trade Flow



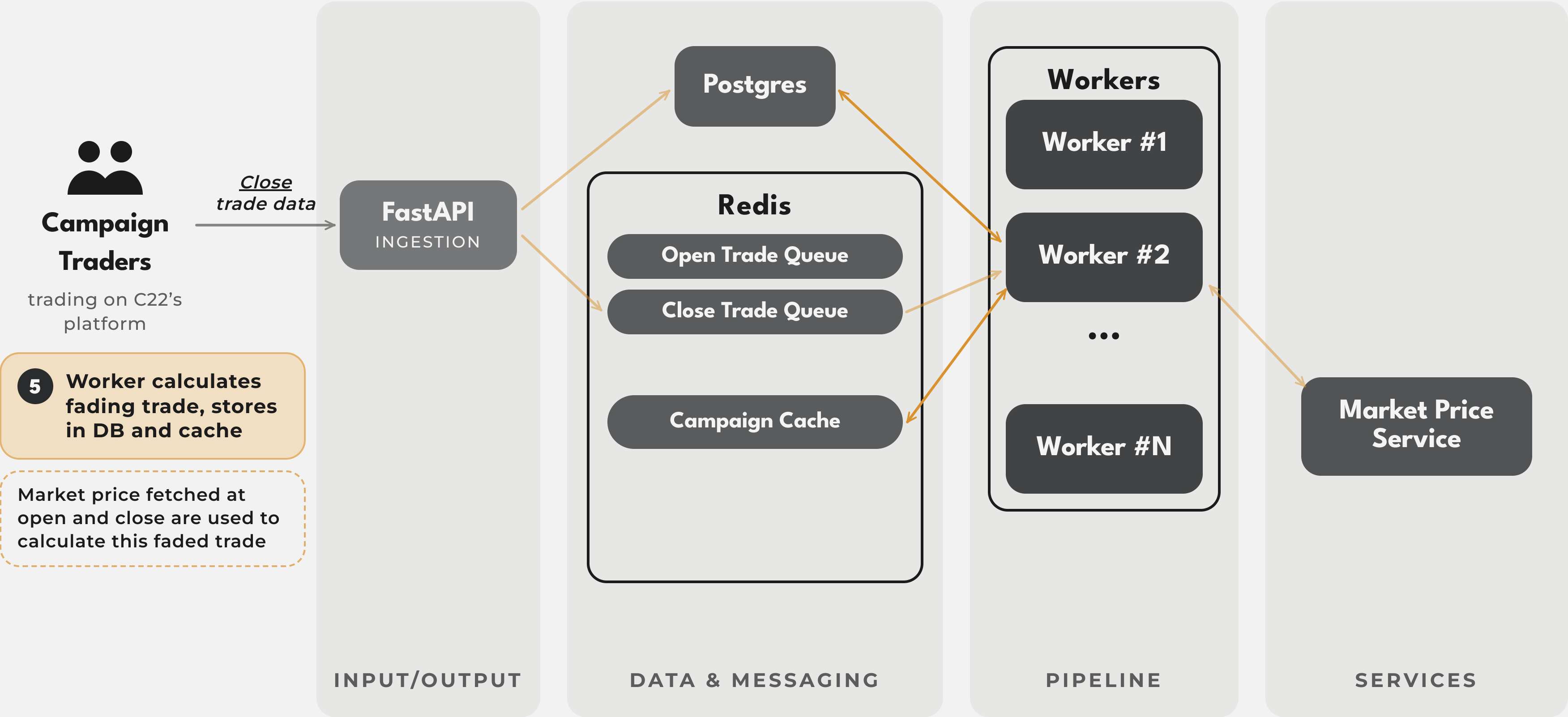
Close Trade Flow



Close Trade Flow

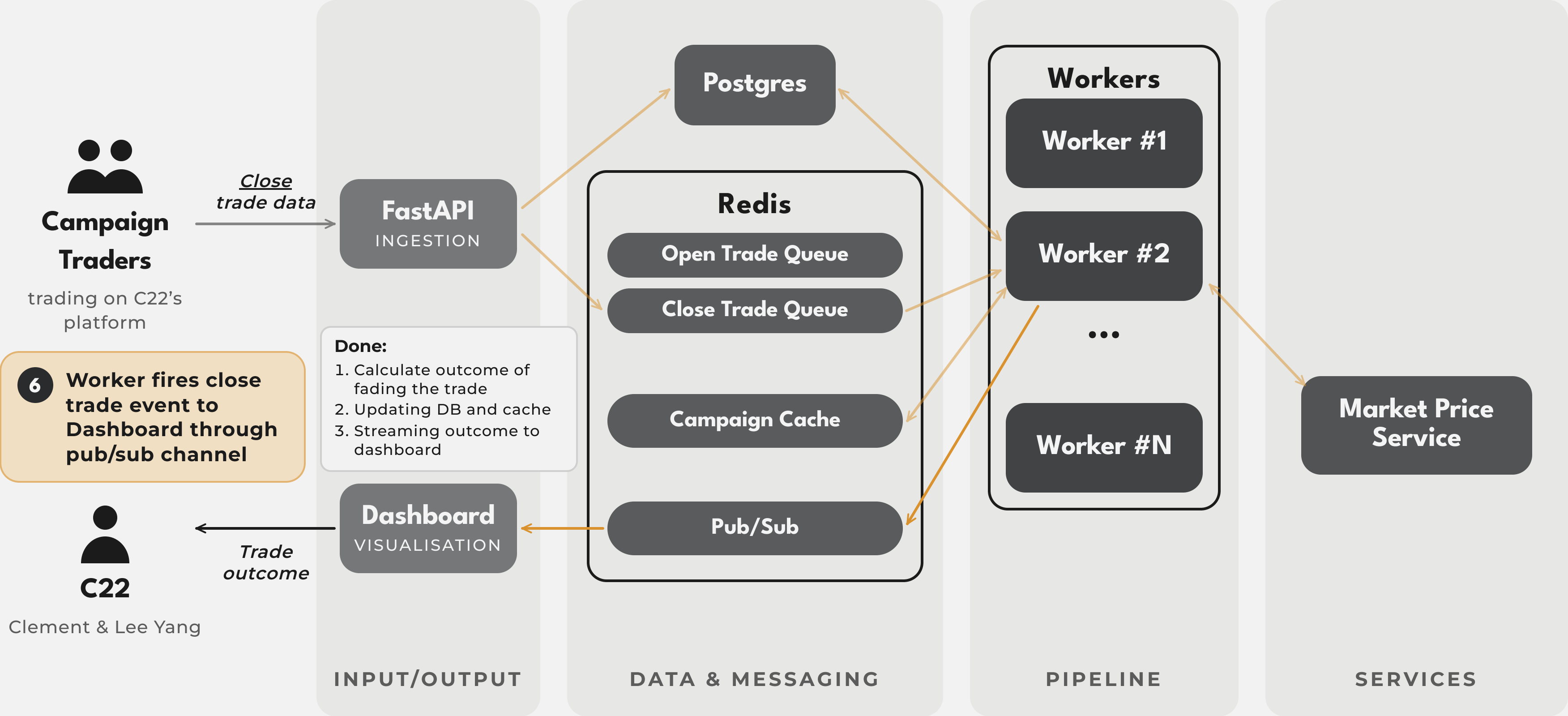


Close Trade Flow



Close Trade Flow

Close trade flow ends



05

Key Design Decisions

The architectural calls we made, and why

DRIVEN BY OUR NON-FUNCTIONAL REQUIREMENTS

Key Design Decisions

Concurrency & throughput



Peak periods send up to 19 trades per minute and 5 per second based on past campaigns.

Sequencing



Trades are processed sequentially (open comes before close).

Processing latency



Median time for price change to exceed acceptable threshold is 0.6-0.85s.

Deployment



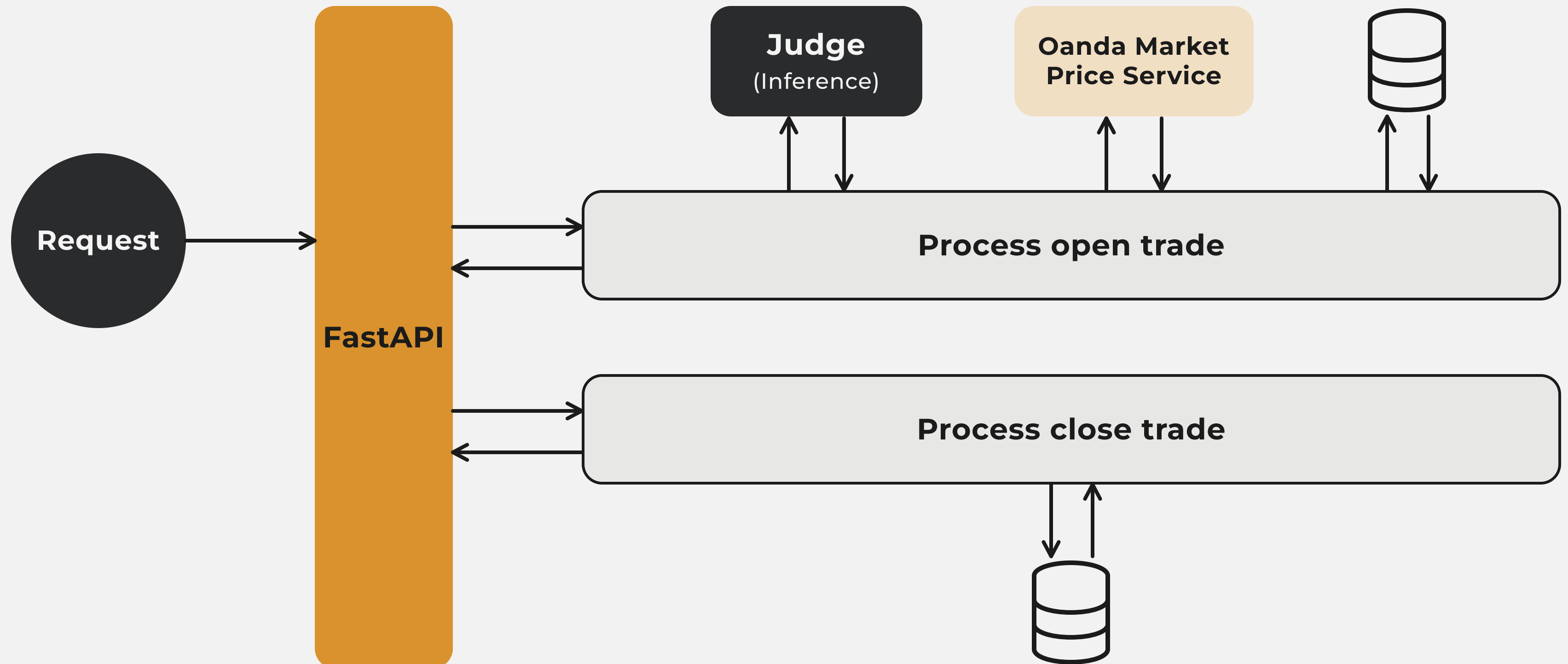
Runs entirely on the sponsor's own hardware with one command.

Process Trades Asynchronously

If it was synchronous...

Concurrency & throughput

⇒ Peak periods send up to 19 trades per minute and 5 per second based on past campaigns.

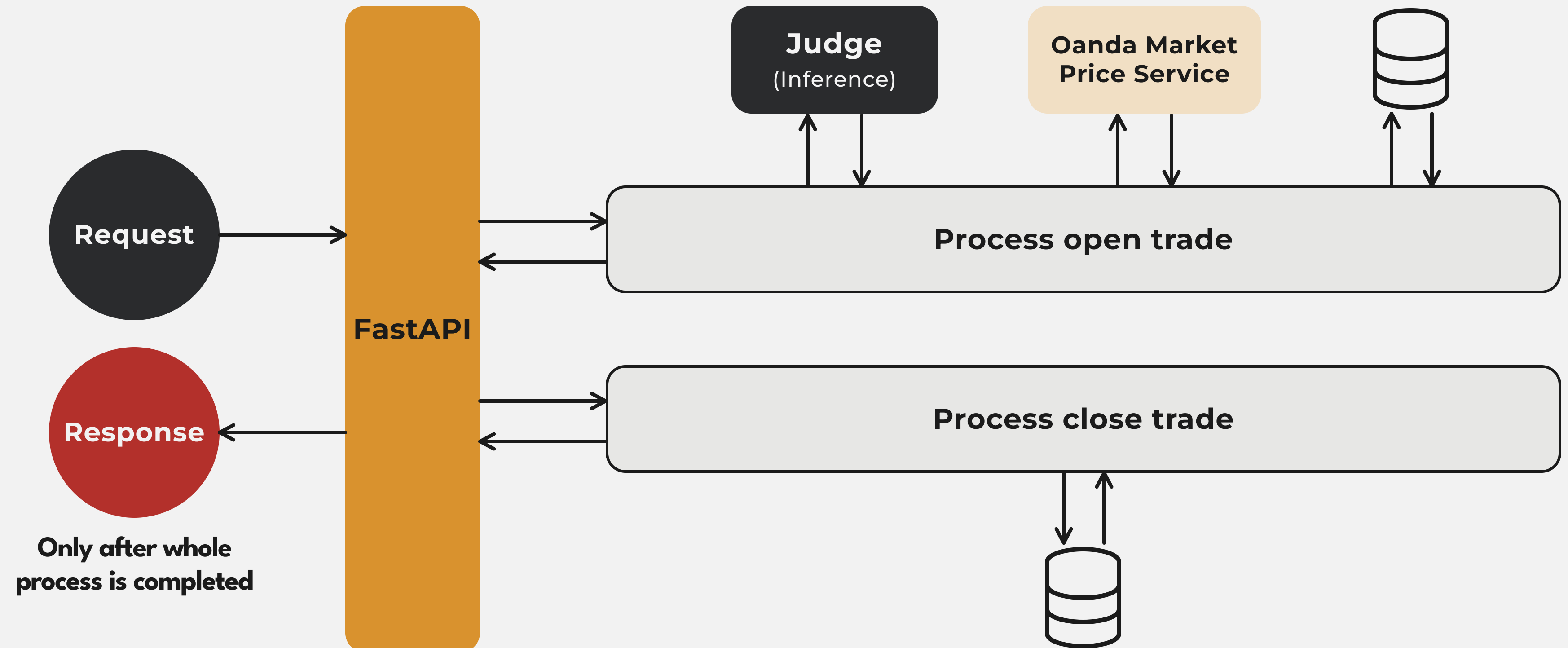


Process Trades Asynchronously

If it was synchronous...

Concurrency & throughput

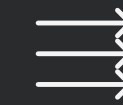
⇒ Peak periods send up to 19 trades per minute and 5 per second based on past campaigns.



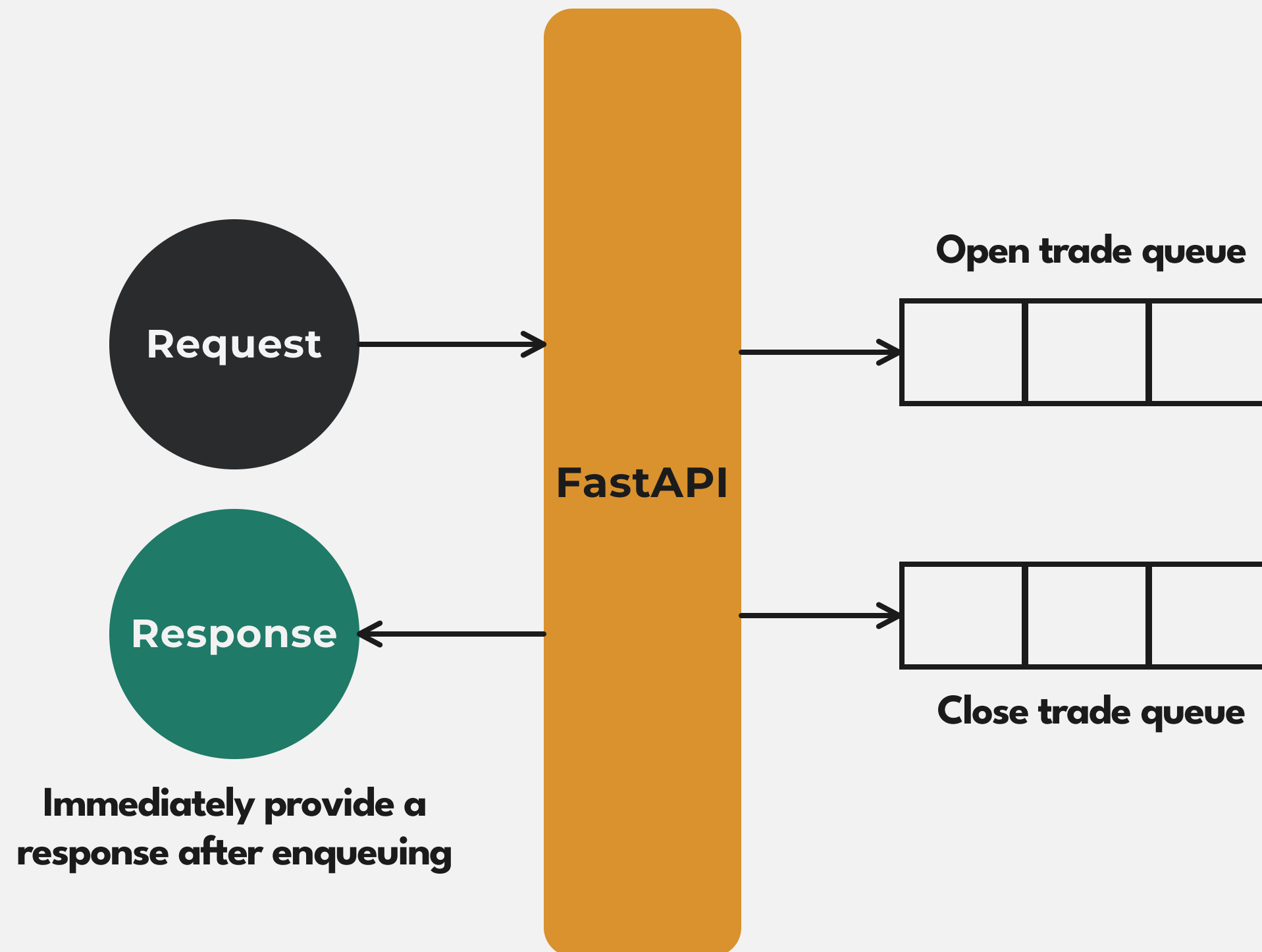
Process Trades Asynchronously

Asynchronous

Concurrency & throughput



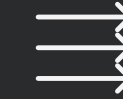
Peak periods send up to 19 trades per minute and 5 per second based on past campaigns.



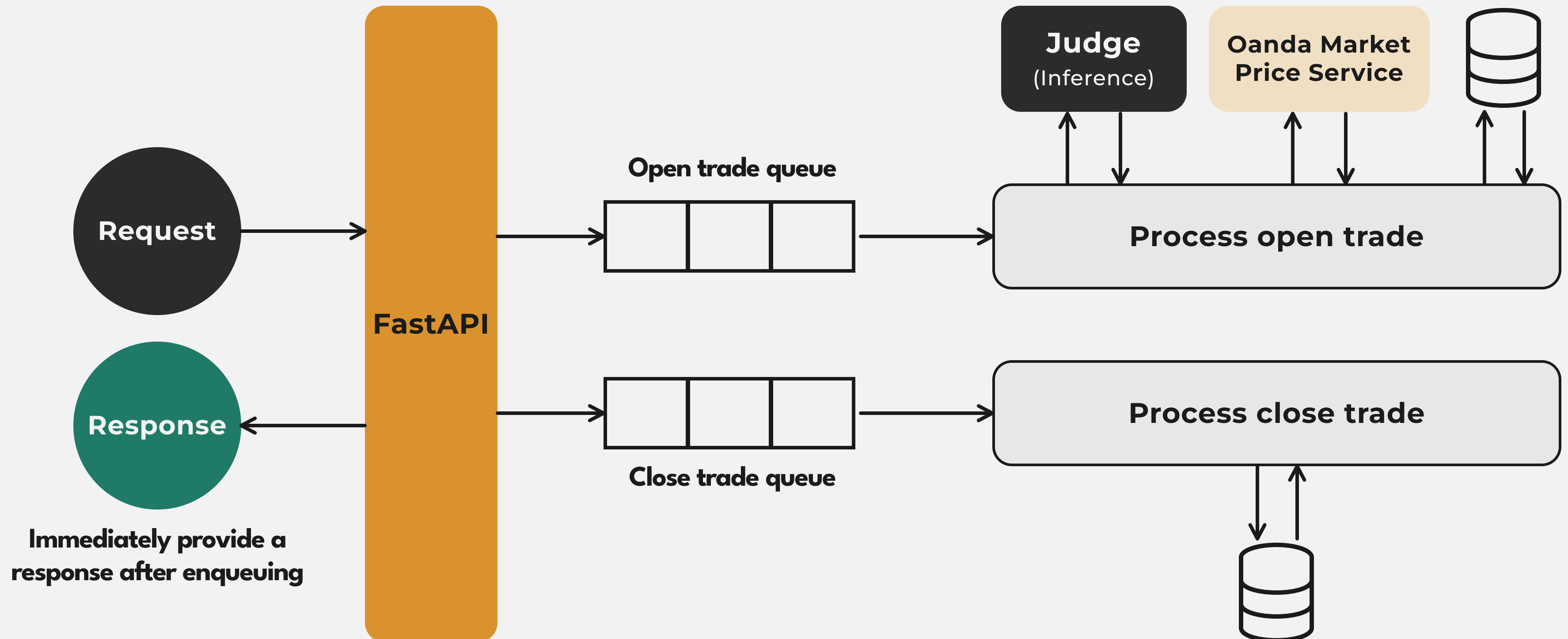
Process Trades Asynchronously

Asynchronous

Concurrency & throughput



Peak periods send up to 19 trades per minute and 5 per second based on past campaigns.



DRIVEN BY OUR NON-FUNCTIONAL REQUIREMENTS

Key Design Decisions

Concurrency & throughput



Peak periods send up to 19 trades per minute and 5 per second based on past campaigns.

Sequencing



Trades are processed sequentially (open comes before close).

Processing latency



Median time for price change to exceed acceptable threshold is 0.6-0.85s.

Deployment



Runs entirely on the sponsor's own hardware with one command.

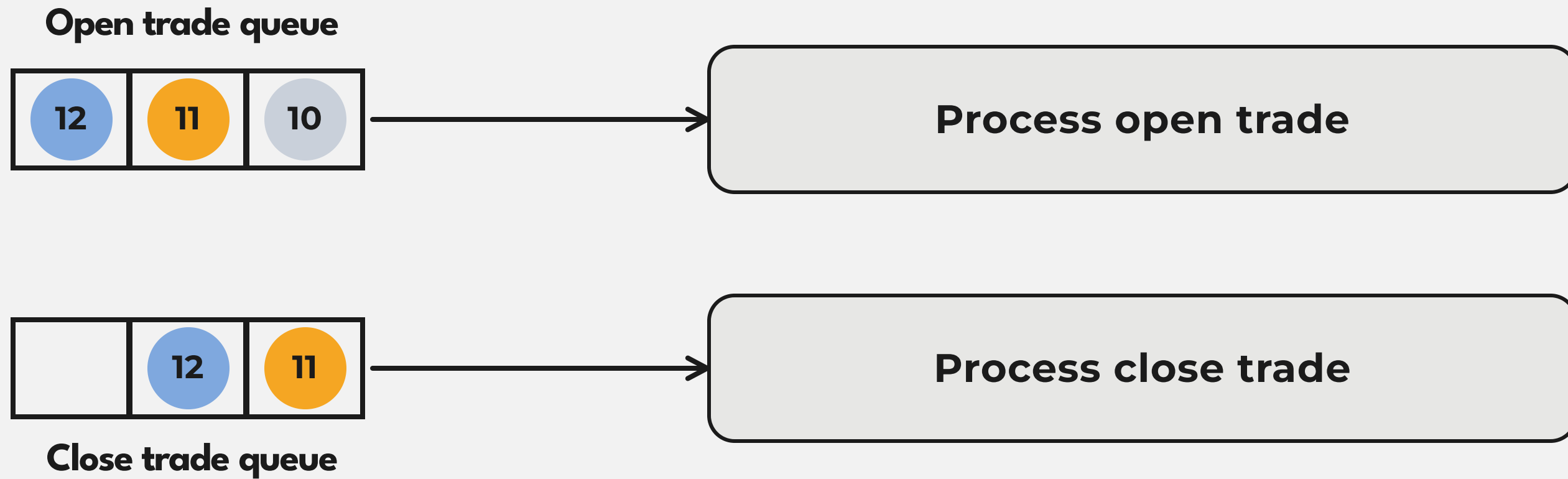
Race Conditions

Close trades processed before its corresponding open trade

Problem

Sequencing

o — Trades are processed sequentially
o — (open comes before close).



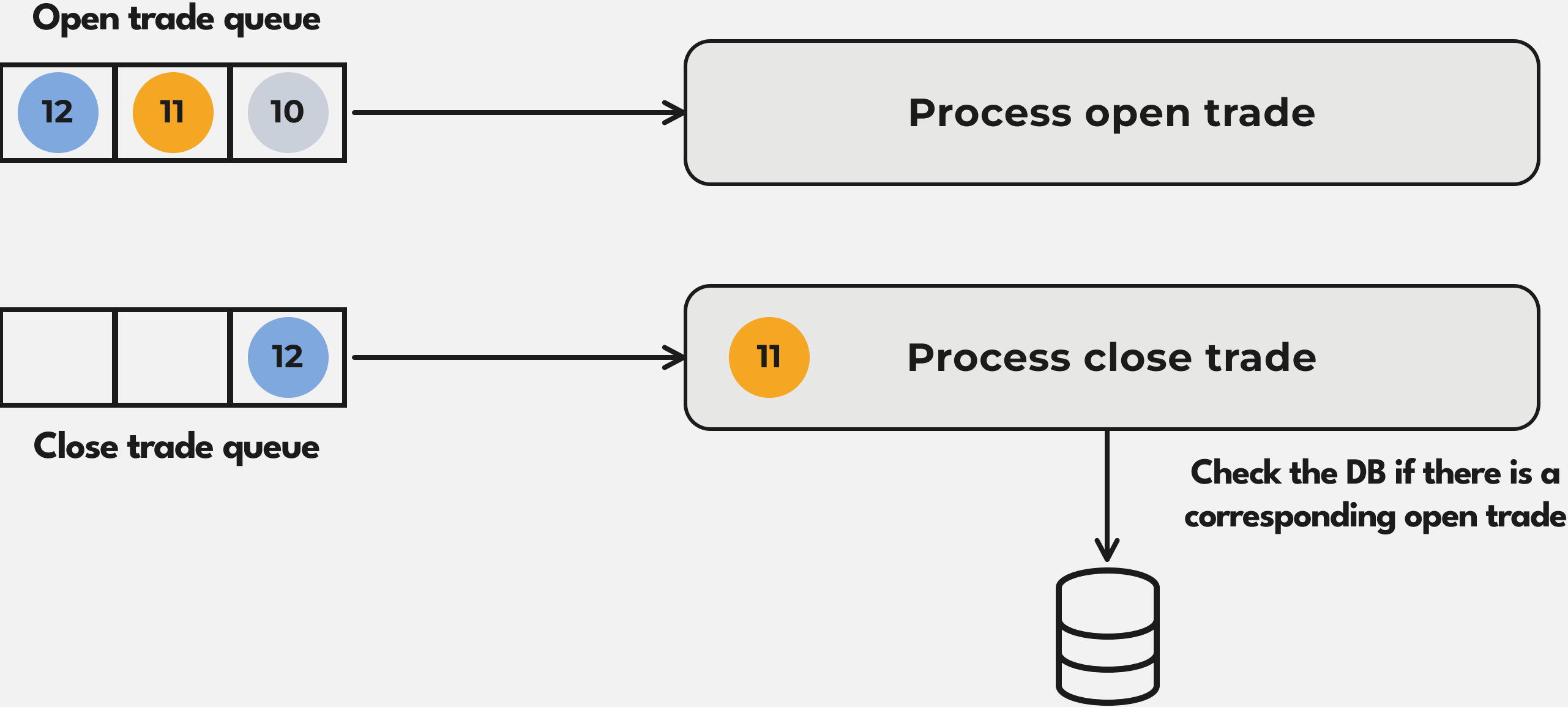
Race Conditions

Close trades processed before its corresponding open trade

Problem

Sequencing

Trades are processed sequentially
(open comes before close).



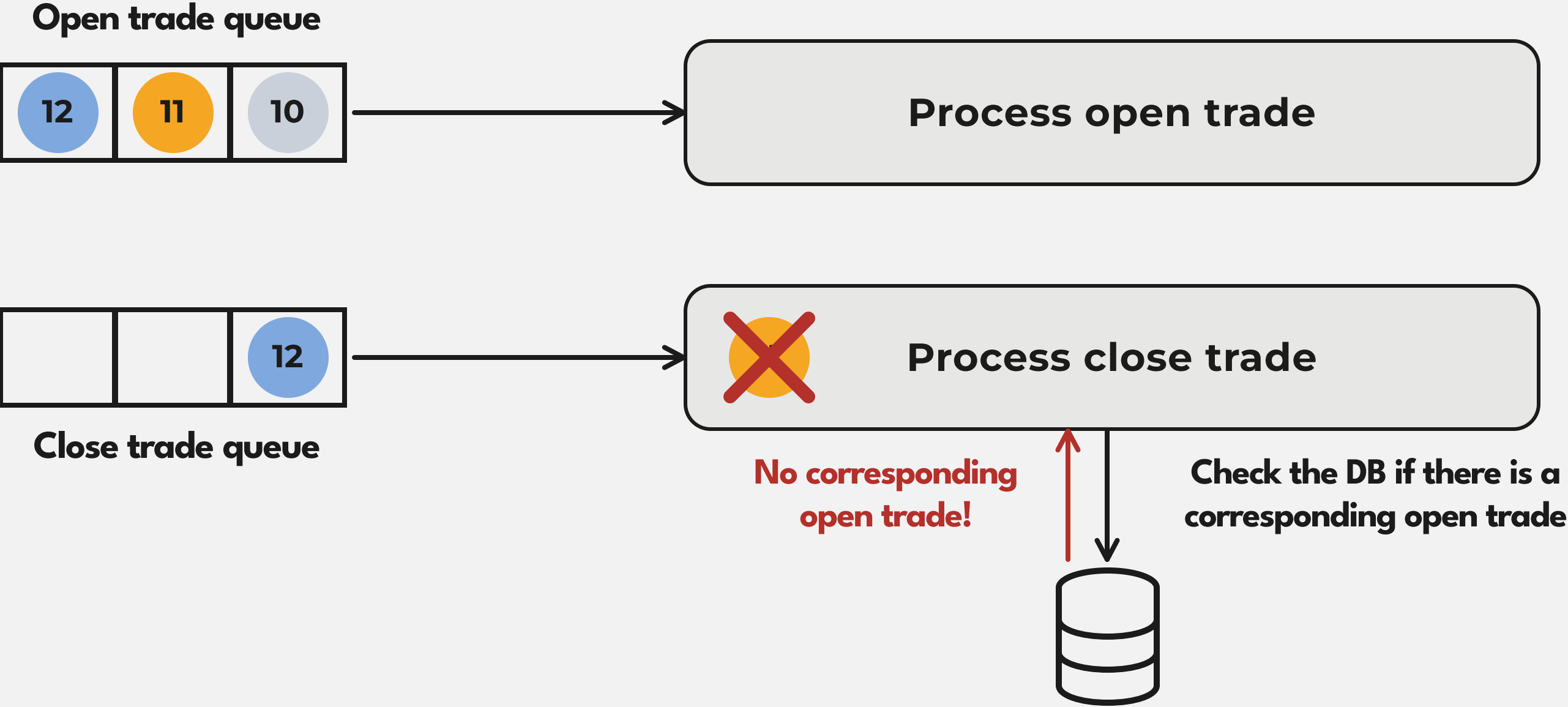
Race Conditions

Close trades processed before its corresponding open trade

Problem

Sequencing

Trades are processed sequentially
(open comes before close).



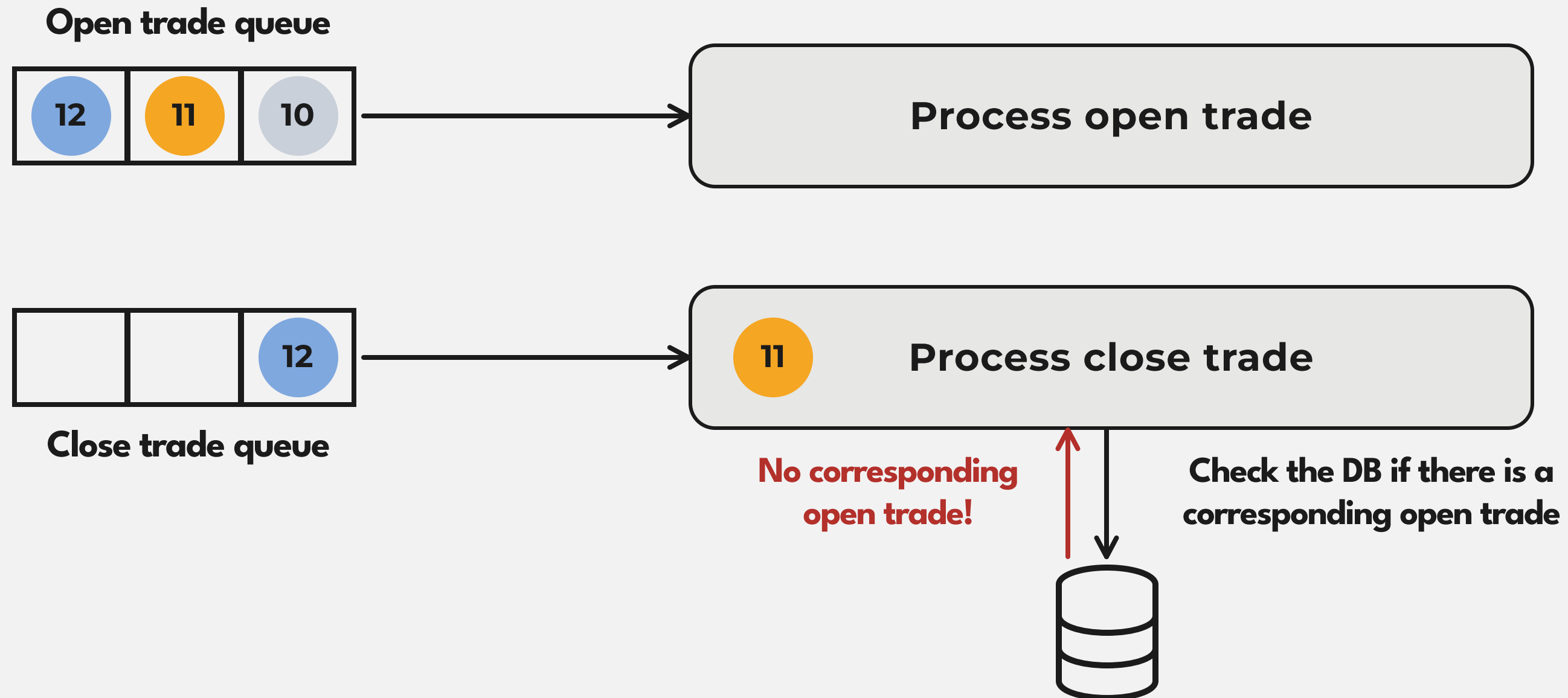
Race Conditions

Close trades processed before its corresponding open trade

Solution

Sequencing

○ — Trades are processed sequentially
○ — (open comes before close).



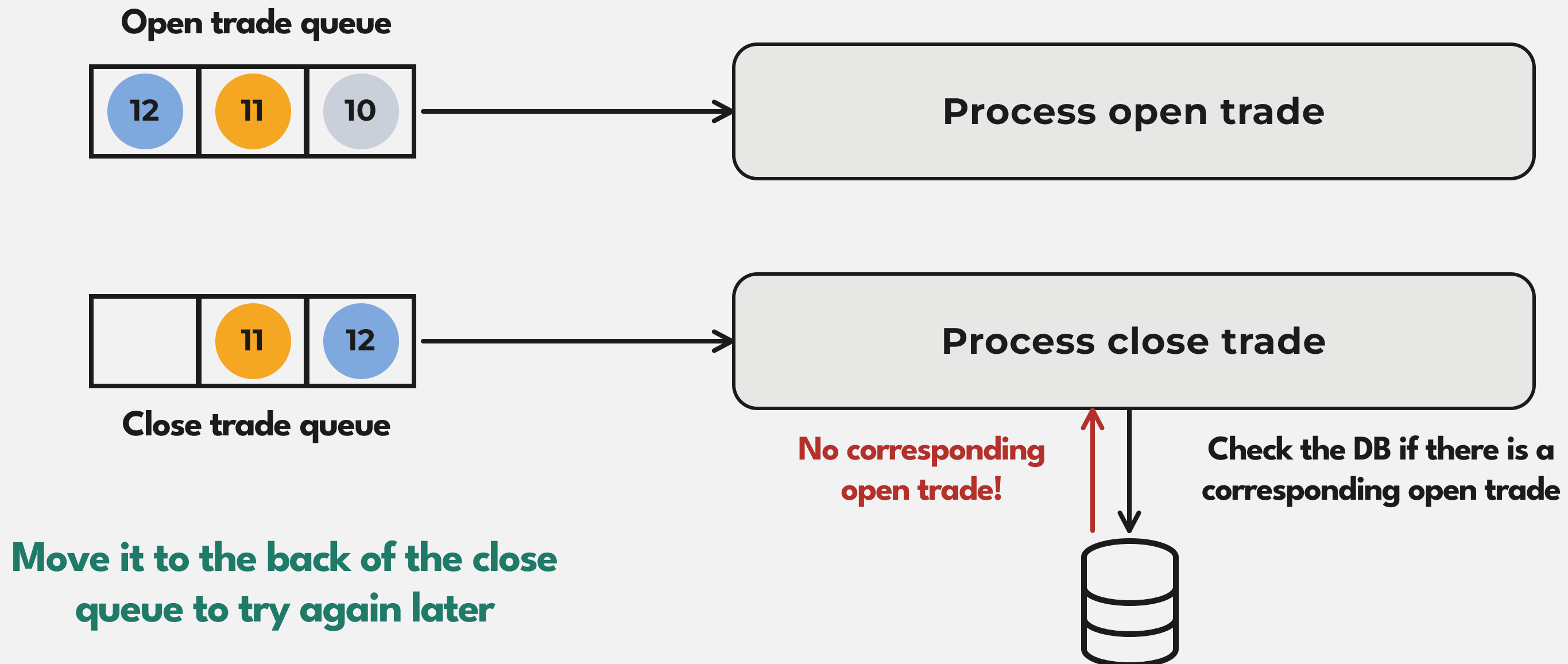
Race Conditions

Close trades processed before its corresponding open trade

Solution

Sequencing

- — Trades are processed sequentially
- — (open comes before close).



DRIVEN BY OUR NON-FUNCTIONAL REQUIREMENTS

Key Design Decisions

Concurrency & throughput



Peak periods send up to 19 trades per minute and 5 per second based on past campaigns.

Sequencing



Trades are processed sequentially (open comes before close).

Processing latency



Median time for price change to exceed acceptable threshold is 0.6-0.85s.

Deployment



Runs entirely on the sponsor's own hardware with one command.

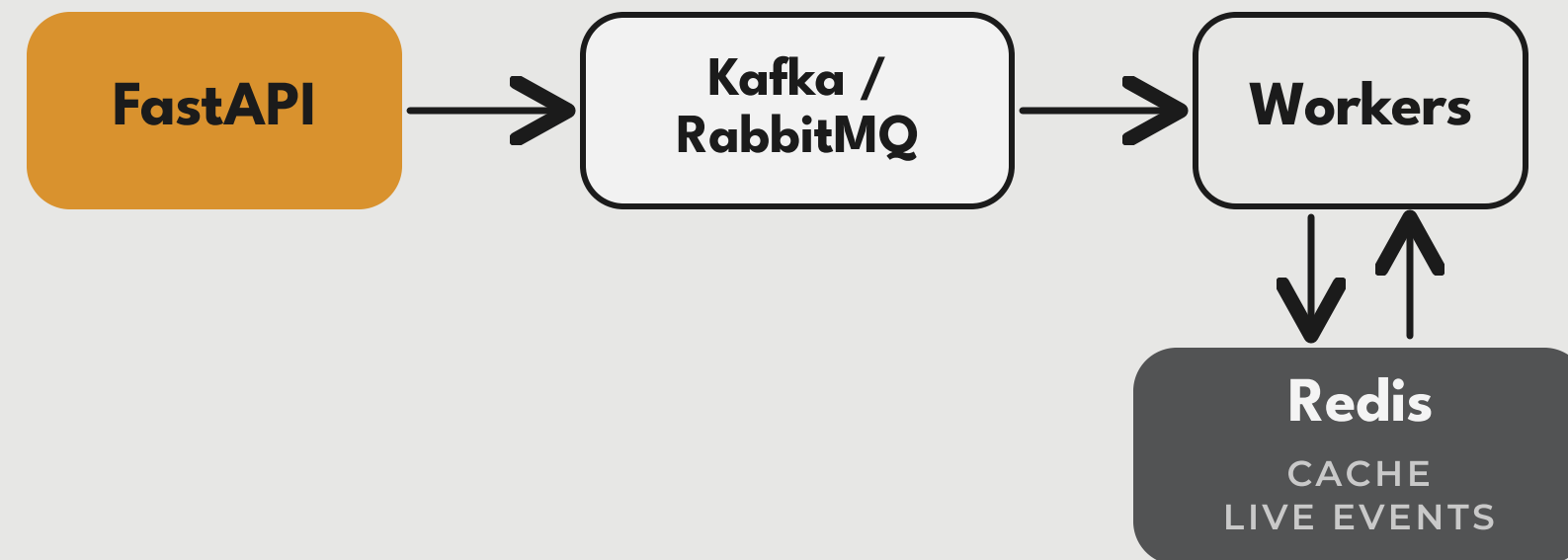
Why Redis?

Processing latency



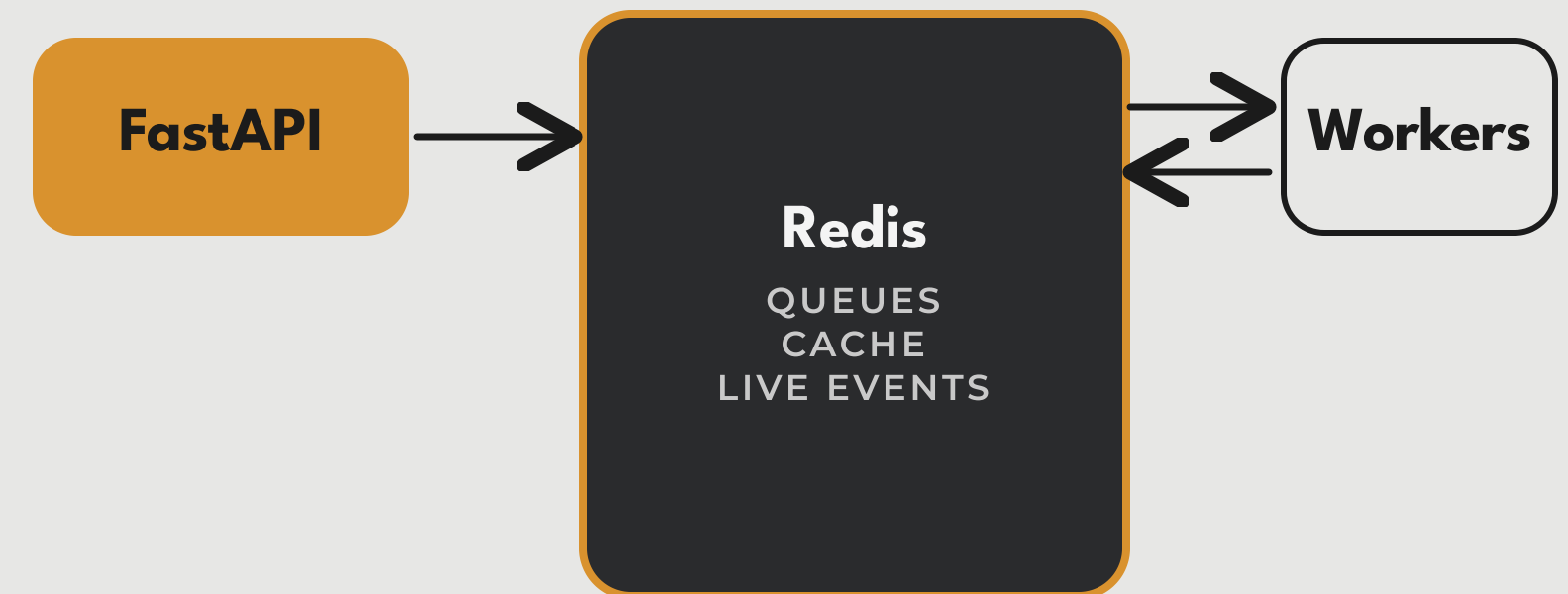
Median time for price change to exceed acceptable threshold is 0.6-0.85s.

WITH A SEPARATE BROKER



2 systems on every trade's path

WITH ONE REDIS



1 in-memory system



Fewer distributed systems

One store for queues, cache and live events: nothing to keep in sync across systems



Less network latency

Each trade waits on one in-memory system, not a broker and a cache

DRIVEN BY OUR NON-FUNCTIONAL REQUIREMENTS

Key Design Decisions

Concurrency & throughput



Peak periods send up to 19 trades per minute and 5 per second based on past campaigns.

Sequencing



Trades are processed sequentially (open comes before close).

Processing latency



Median time for price change to exceed acceptable threshold is 0.6-0.85s.

Deployment



Runs entirely on the sponsor's own hardware with one command.

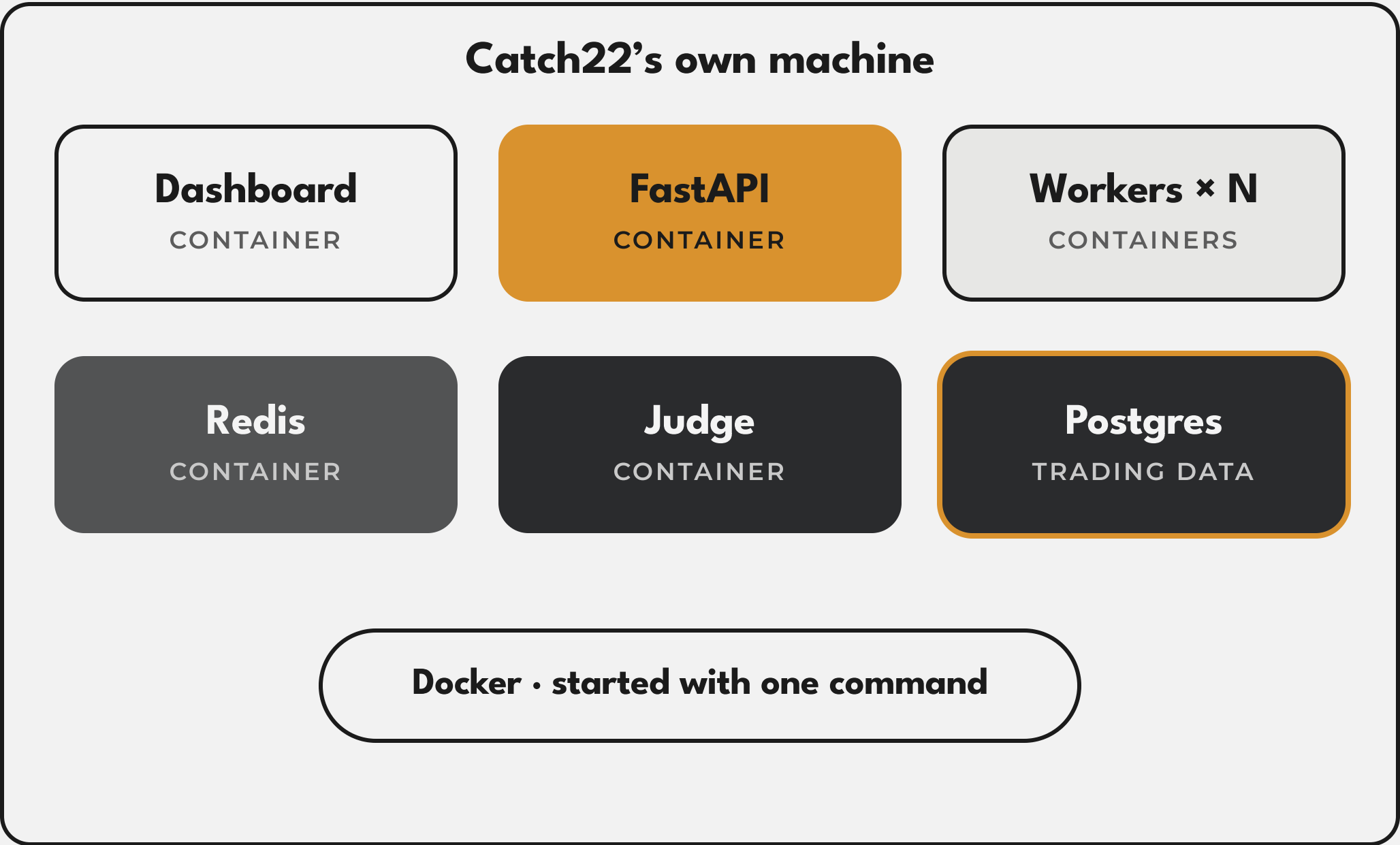
Dockerised, Deployed On-Site

Deployment
 Runs entirely on the sponsor's own hardware with one command.

**Sponsor's hardware**
They have the machines and the know-how

**Data stays on-site**
Sensitive trading data in their own database

**No extra cost**
No monthly cloud bill



Only outside call: OANDA market prices



How We Got Here

Training, features and choosing the final model

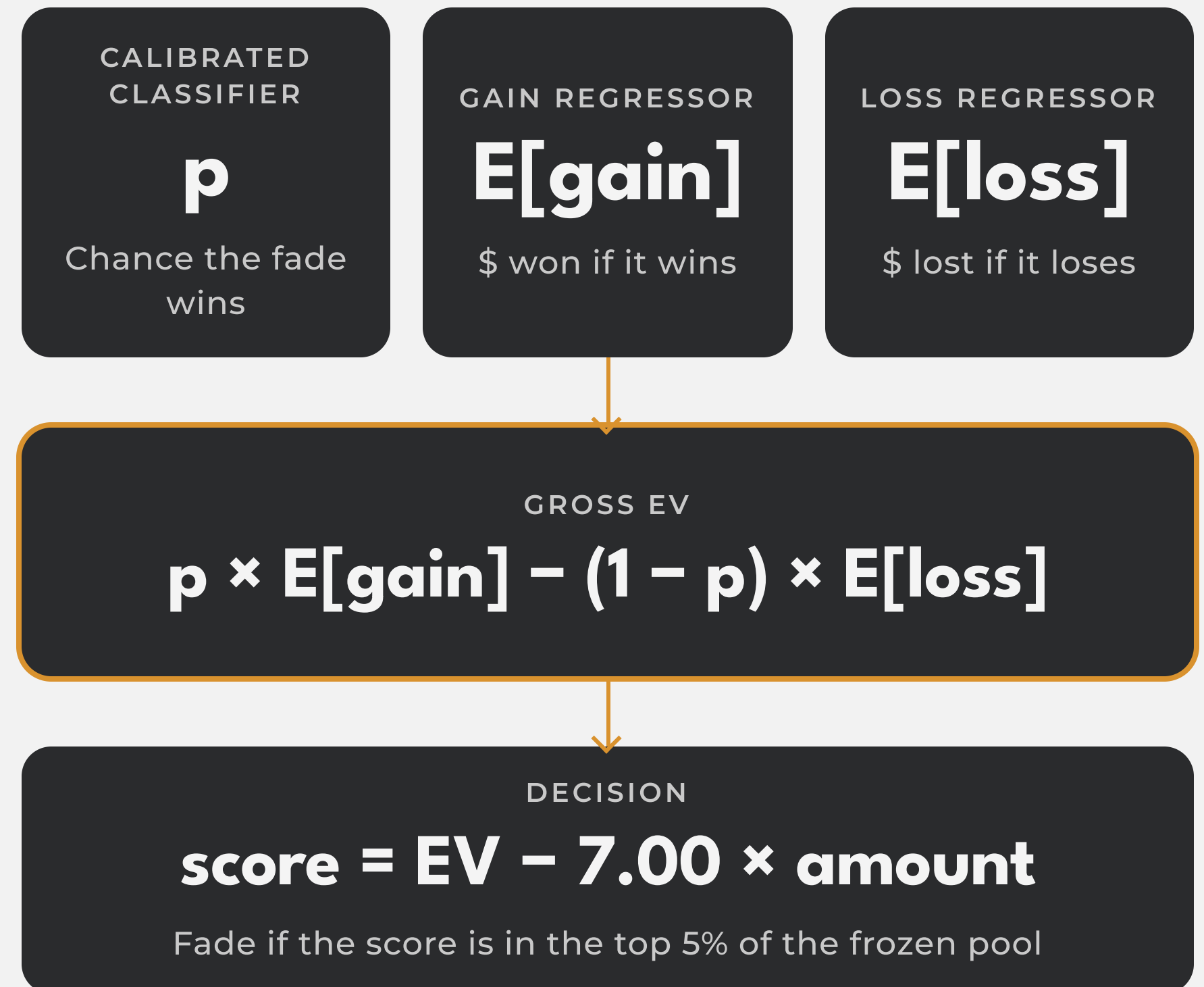
An ensemble of LightGBM models

In one sentence

Three small models. Each fills in one term of the expected-value equation.

Why it fits

- Tabular data
- Small dataset
- Built-in regularisation
- Fast inference



HOW WE GOT HERE

Our Approach to Training

STEP 01

Data

45 campaigns,
68,377 trades

STEP 02

Features

26 point-in-time
features

STEP 03

Train

Hurdle: $P(\text{win}) \times$
gain – loss

STEP 04

Validate

Locked test vs
baselines and luck

How the Data was used

54

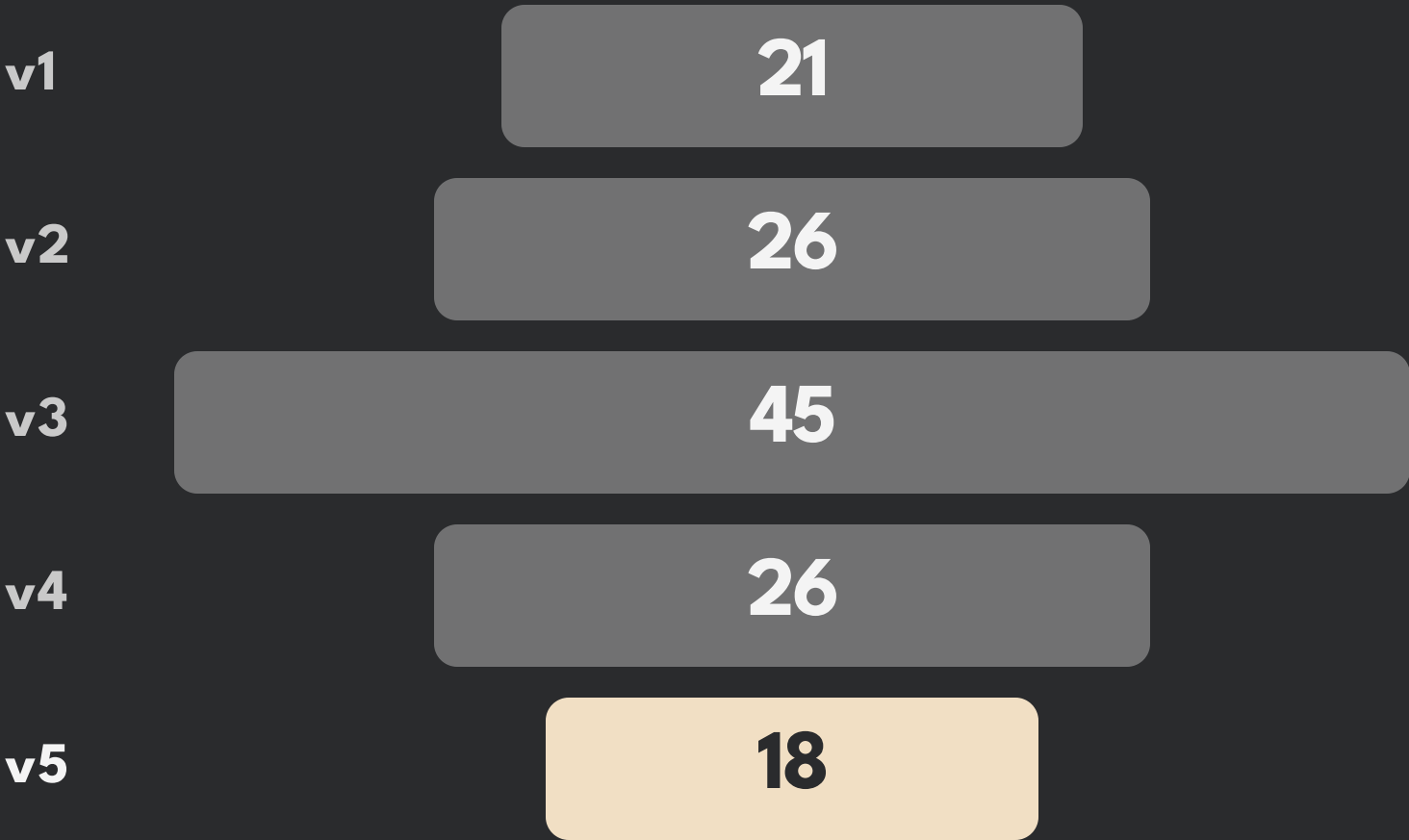
Campaigns from C22

Every customer trade is a row of 28 fields with information on trader's side, amount, open and close prices, and open and close times

Campaigns

33–68	Training set
69–77	Testing set
78–86	Additional test set

How the feature count changed



Our final feature set, frozen for judge model

Raw Fields We Ruled Out

6

No variance

Same value on every trade

Same contract

`instrument = XAUUSD`

`currency = USD`

`lotSize = 100`

Never charged

`swap = 0`

Cross price always 1

`openTradeCrossPrice`

`closeTradeCrossPrice`

7

IDs & identity

Teaches who trades, not
how they trade

Trade and order IDs

`positionId, closeTradeId`

`openOrderId, closeOrderId`

Account

`accountId`

Personal data

`EMAIL, IP_ADDRESS`

4

Near-duplicates

Spearman correlation
 $r_s > 0.8$

Fee tracks size

`commission ↔ amount`

$r_s = -1.00$

Both follow gold's trend

`openPrice ↔ campaignId`

$r_s = -0.86$

2

Outcome leakage

The trade's result, so
never an input

Profit

The reverse of the reverser's
profit

Profit after costs

`netProfit`

`= profit + commission + swap`

The 13 Features We Dropped

6

Leakage

Information the model won't have at open trade time

Stop-loss or target set?

has_sl, has_tp

Distance to stop and target

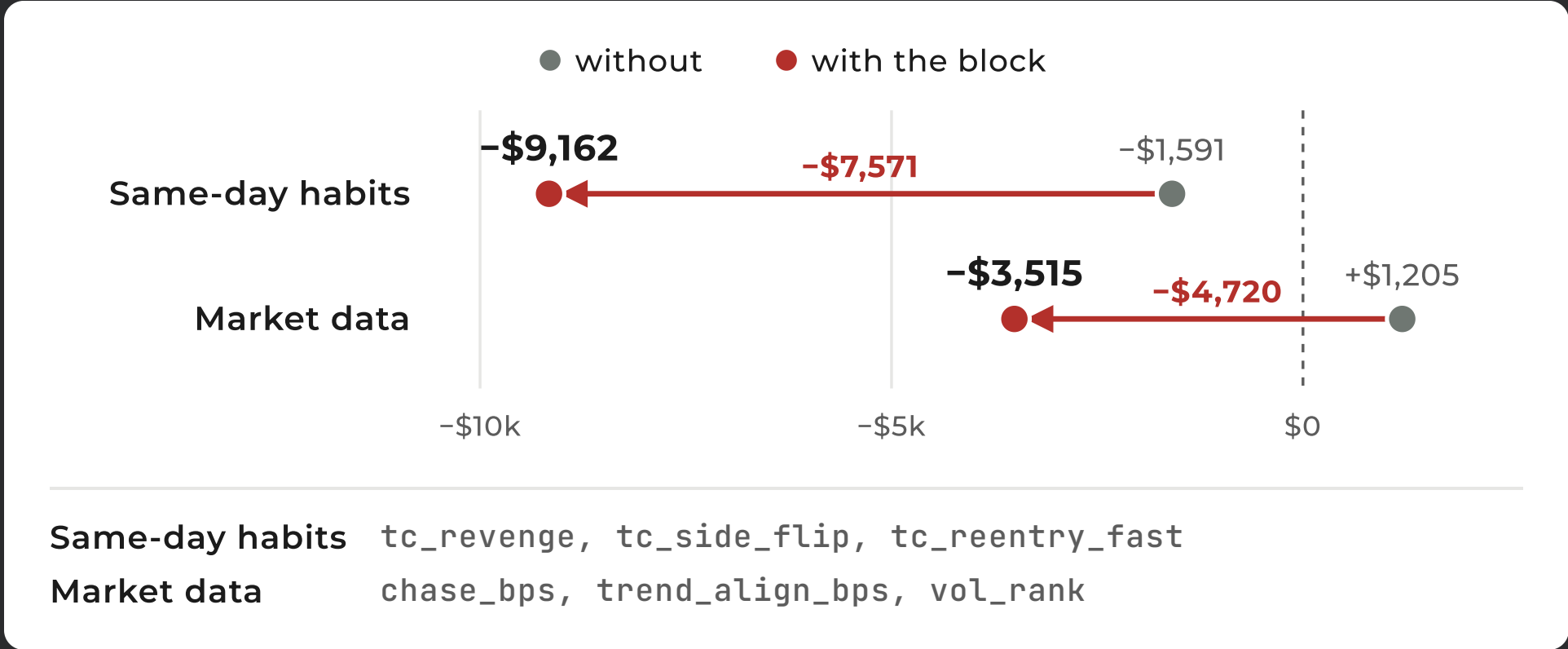
sl_dist_bps, tp_dist_bps, implied_rr

Stop moved past entry

sl_beyond_entry

7 Fair, but didn't pay

Profit from our top-5% bets, 9 campaigns the model never saw



Predicted stop-loss p_sl

Overlapped 90% with “stop-loss habit”, a feature we already had. Added nothing new.

Why We Left Out Outcomes

8

Outcome features

How their earlier trades turned out, not how they trade

Win rate today

prior_win_rate

Win rate on past days

prior_campaign_win_rate

Profit so far today

cum_prior_profit

Profit on past days

cum_prior_campaign_profit

Current win/loss streak

current_streak

Drop from today's best

running_drawdown

Stops actually hit

prior_stop_hit_rate

Targets actually hit

prior_target_hit_rate

Using SHAP

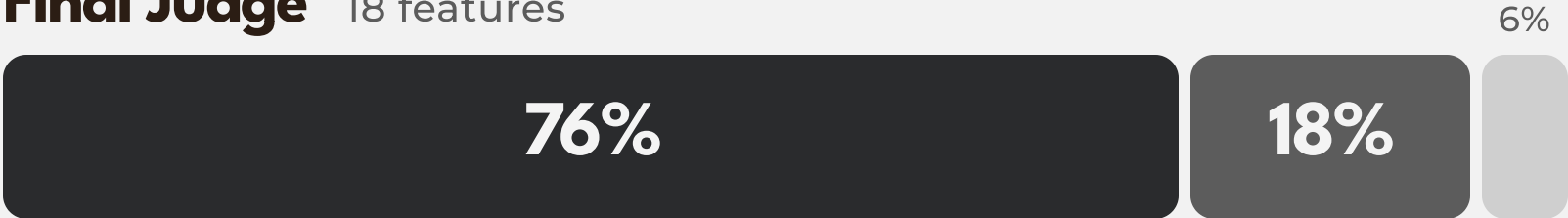
SHAP shows each feature's share of the model's decision.

Full model 26 features



↓ Remove the 8 outcome features

Final Judge 18 features



- Outcome
win rate, streak, P&L
- Behaviour
stops, sizing, re-entry
- Order ticket
side, size, time
- Counters
trade counts

Share of the model's attention (mean |SHAP|). Source: Notebook 12.

The 18 Features We Kept

6

The trade itself

Side, amount, hour given; others derived

Buy or sell

side_is_buy

Trade size

amount

Hour opened

hour_utc

Time on a clock face

tod_sin, tod_cos

Gold's move today

price_vs_campaign_open_bps

8

Earlier today

From the customer's finished trades today

Trades so far today

n_prior_trades

Size vs their usual

escalation_ratio

Usual hold time

mean_prior_duration

Take-profit habit

prior_tp_use_rate

First trade of the day

is_cold_start

Wait since last trade

sec_since_last_close

Stop-loss habit

prior_sl_use_rate

Usual stop distance

mean_prior_sl_dist_bps

4

Earlier days

From campaigns already finished

Past days traded

n_prior_campaigns

Trades on past days

n_prior_campaign_trades

Days since last traded

days_since_last_campaign

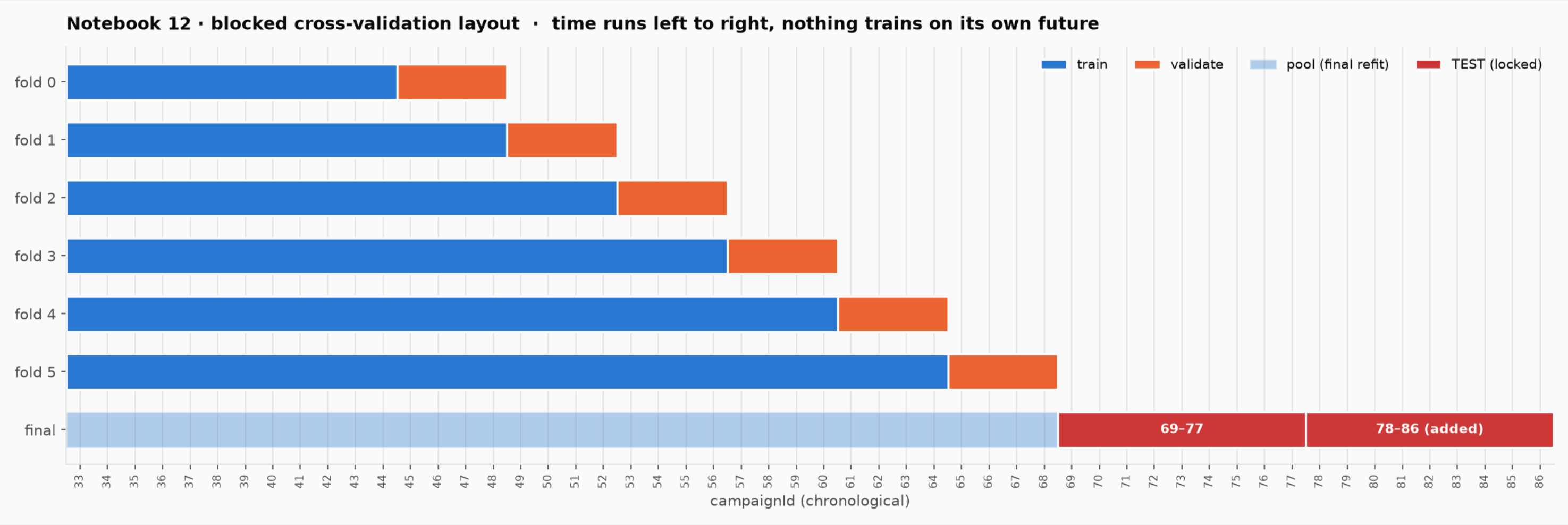
Past stop-loss habit

prior_campaign_sl_use_rate

How We Cross-Validate

BLOCKED CV · CAMPAIGNS 33-86

Six expanding folds, then a locked test block

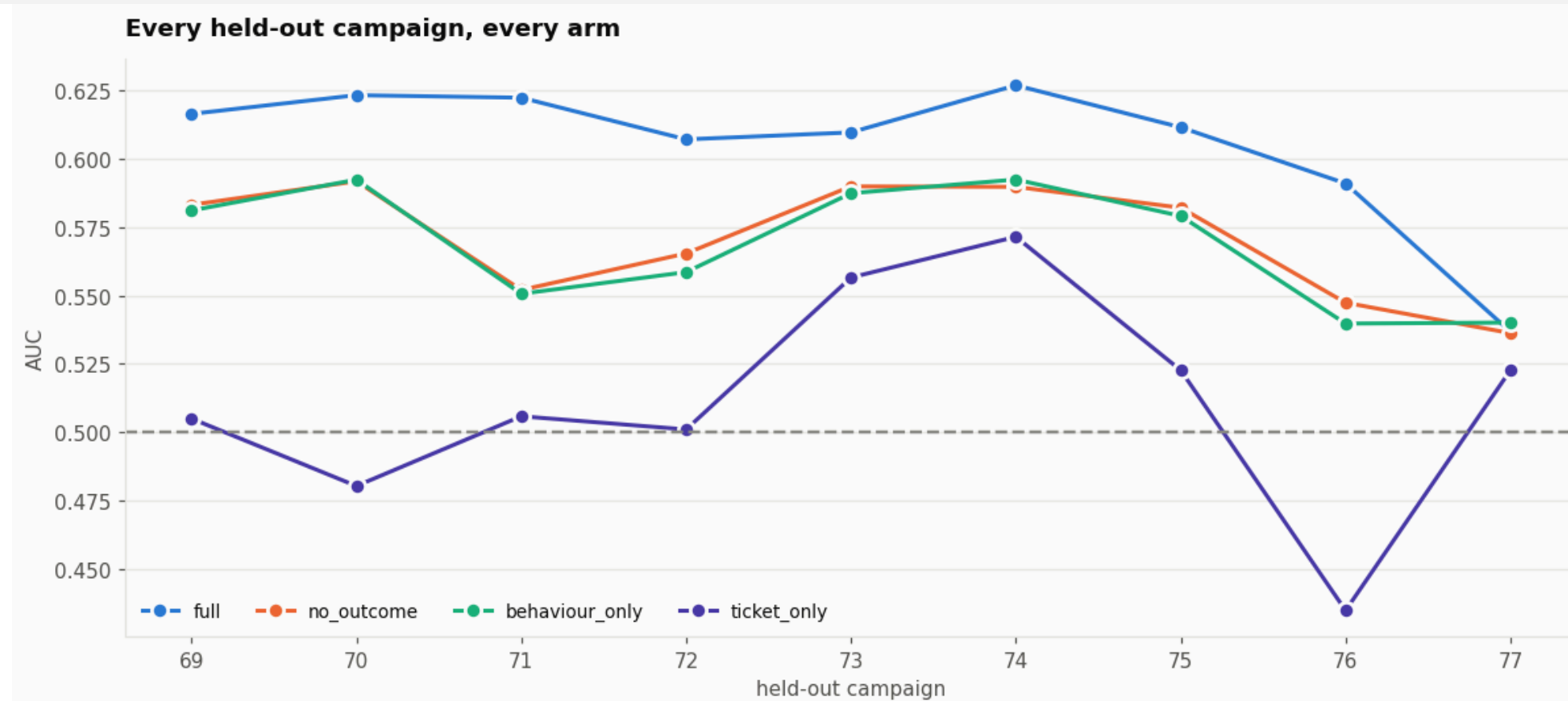


Time runs left to right
Nothing trains on its own future

Every Arm, Every Campaign

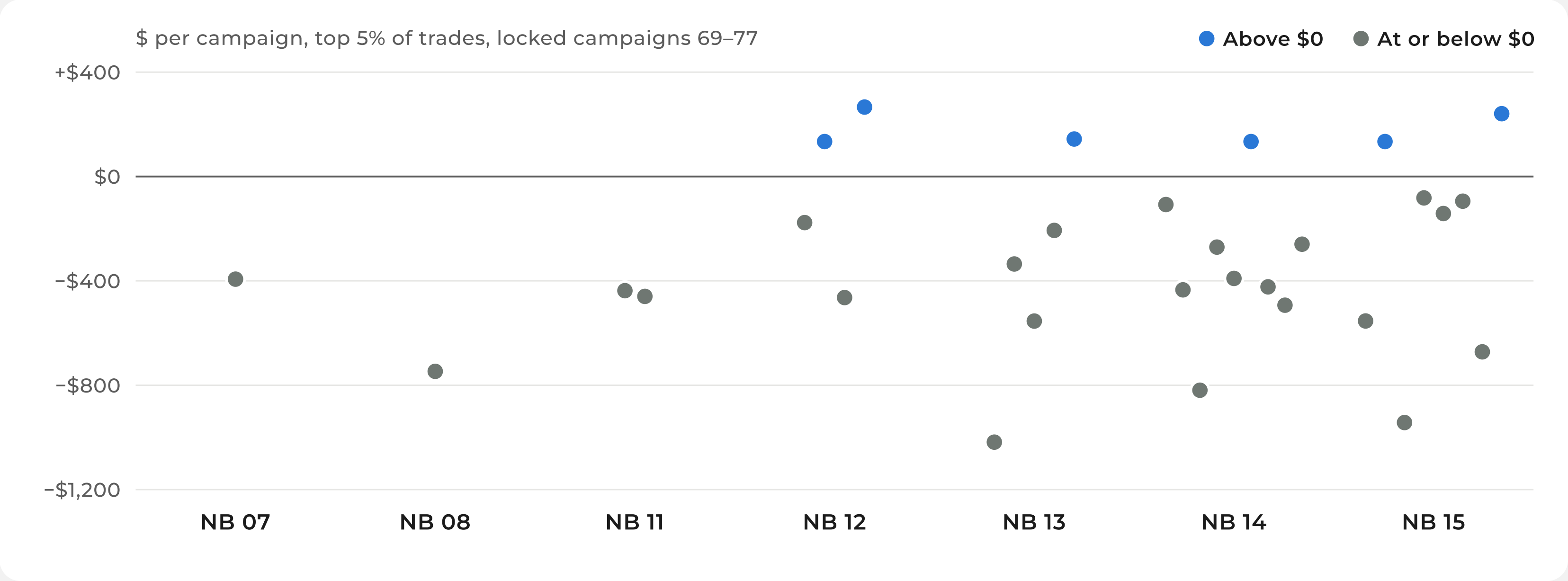
HELD-OUT AUC · CAMPAIGNS 69-77

Feature-set ablation



Removing outcome history hurts in 9 of 9 campaigns
Ticket size alone sits near chance

What Each Notebook Tried



Only 6 of 30 arms finish above \$0.
Where we have intervals (NB 13–15), every one spans zero.

Where the Signal Lives

35x

Perfect regressors vs a perfect classifier

+\$12,417 vs +\$351 per campaign at the top-5% readout

Tried and dropped

What did not pay

Lever	Result
Market data	Rejected 4x
Tuned GBDT vs logistic	+0.002 AUC
More campaigns	Flat curve

07

Project Management

How we ran the work

Why Scrum over Waterfall

Waterfall

Plan everything, then build

Requirements

Design

Build

Test

Deliver

SPONSOR FEEDBACK
Once, at the end

Scrum

What we ran: 2-week sprints

1

2

3

4

5

6

7

8

MIDTERM

🔁 plan · build · review · retro, every sprint

SPONSOR FEEDBACK
Every 2 weeks

1

Requirements weren't fixed

Features to explore, dashboard to be decided later on.

2

Results set the next step

Each sprint's model scores affects what we tried next.

3

Inputs arrived late

Sponsor data and API access given towards the end of the project.

Project Timeline

DELIVERABLES



PHASES

JUN

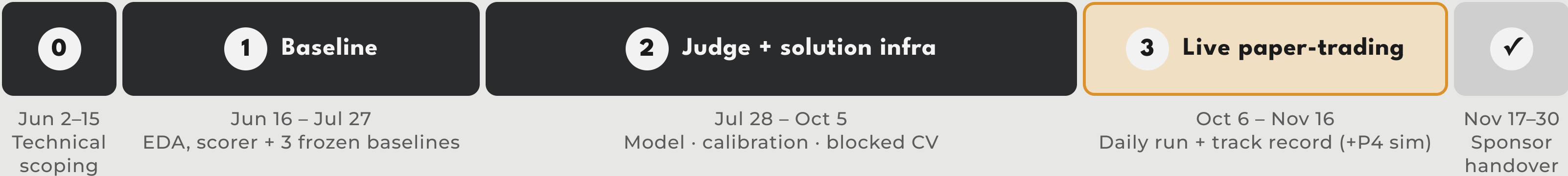
JUL

AUG

SEP

OCT

NOV



Phase 0

Logging live · ID join confirmed

Phase 1

3 baselines frozen pre-training

Phase 2 · today

Judge beats the rules-based system

Phase 3

Credible live track record

Project Milestones

Midterm

Delivered today

Reverse Trade Judge



Model and solution infrastructure

Analytics Dashboard



Model performance and overall metrics

Final

Still to come

Paper Trading



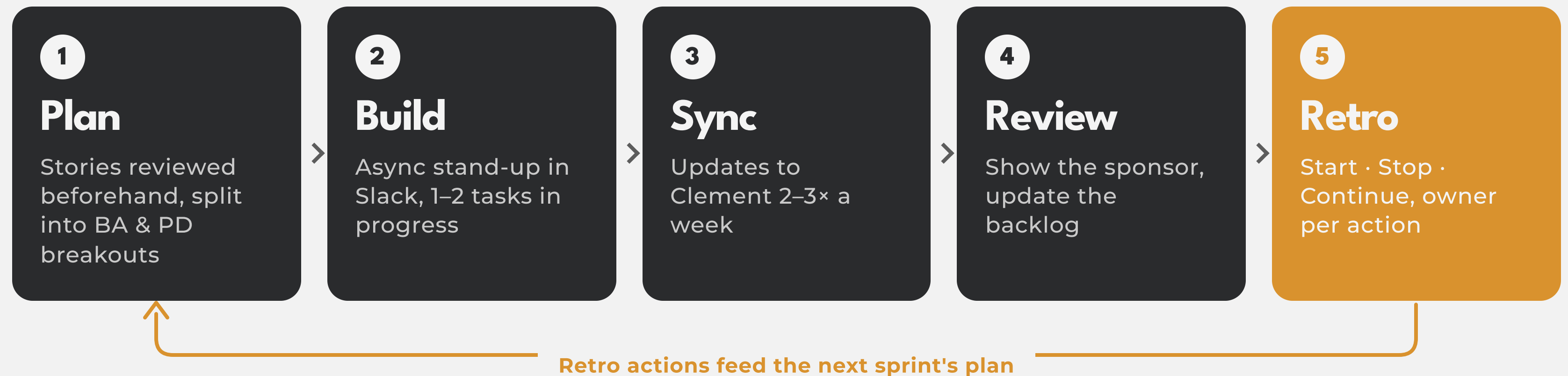
Live paper trading simulator

Auto-Retraining



Automated retraining model

How We Run a Sprint



What Our Retros Changed

WE NOTICED

Vague user stories, so we read requirements differently

Too much in progress, work left to the last minute

unsynchronized updates to each other and the sponsor

WE CHANGED

Review stories before planning; BA & PD plan in breakouts

Max 1–2 tasks in progress; one change per branch

7 pm Slack stand-up reminder; Clement updated 2–3× a week

205

story points completed

What Limits the Results

18

Held-out campaigns

Our final test set. Small AUC gaps can't be told apart

1 Not a random sample

Campaigns and traders weren't randomly chosen, so results may not generalise.

2 Not fully representative

C22 sets the rules: one trade open at a time, accounts first come, first served. Not randomly chosen.

3 Only ~6 months of data

Too short a range. Trades are correlated, so samples aren't independent.

How Big a Test Do We Need?

4x

More campaigns each time the edge halves

The smaller the edge we care about, the longer it takes to confirm

Campaigns to confirm an edge

Edge = average profit per faded trade, after costs. Bigger edges stand out from noise faster, so they need fewer campaigns. Our 18 test campaigns can reliably detect edges of ~\$10 a trade or more.

Edge / trade	Campaigns	Worth per month
\$1.32	840–1,330	\$1,200
\$3	160–260	\$2,700
\$5	60–90	\$4,500
\$10	15–23	\$9,000

What's Next

NEXT 01

Auto-retrainer

Retrain the Judge automatically as new data lands

NEXT 02

Real pipeline

Wire the Judge into C22's live platform API

NEXT 03

Trade magnitude

Go beyond fade / pass to sizing each fade

NEXT 04

Auto-scaling

Workers scale with campaign load

NEXT 05

Non-functional reqs

Latency, throughput and the rest

NEXT 06

Observability

Prometheus and Grafana, identify bottlenecks

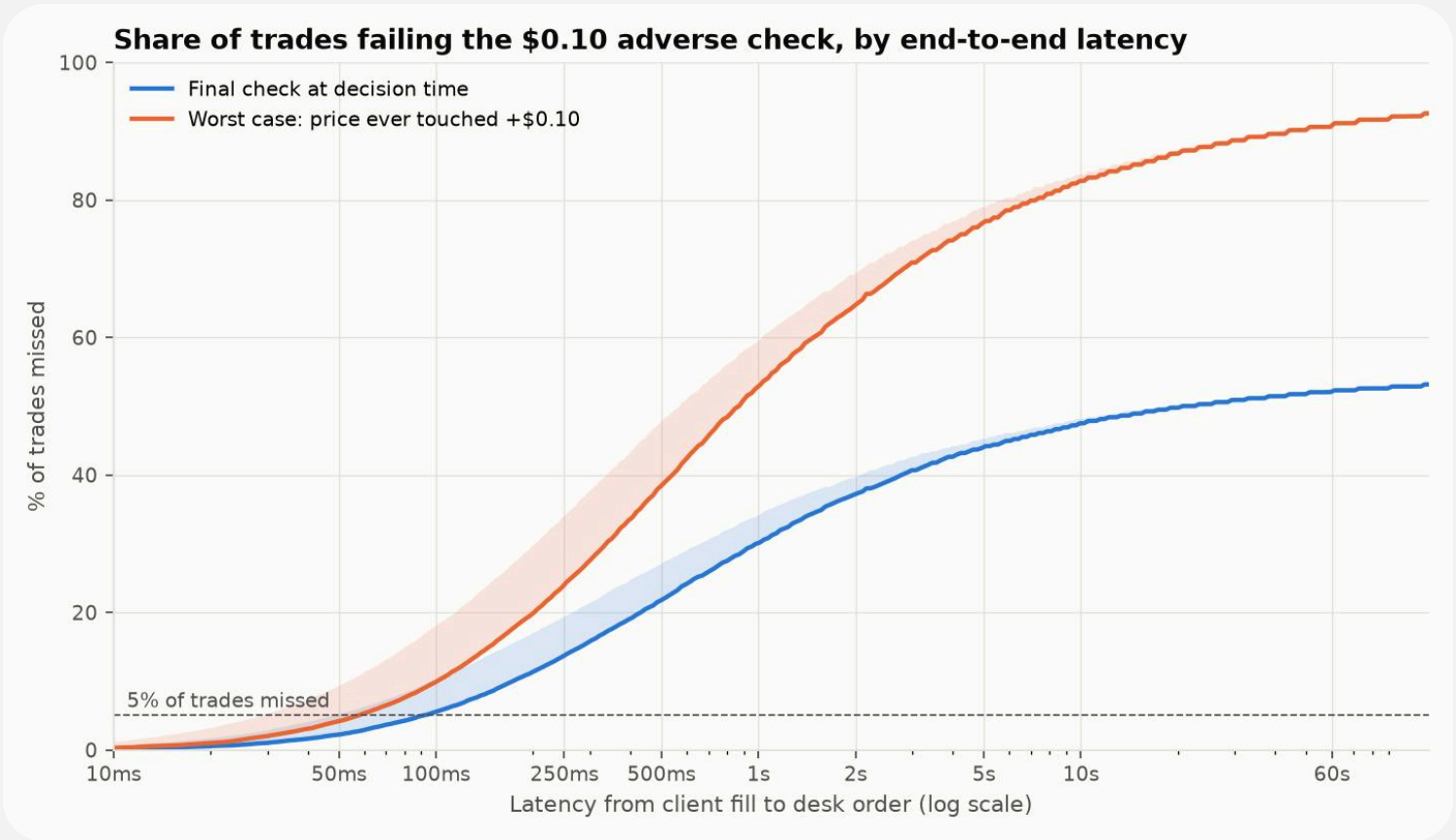
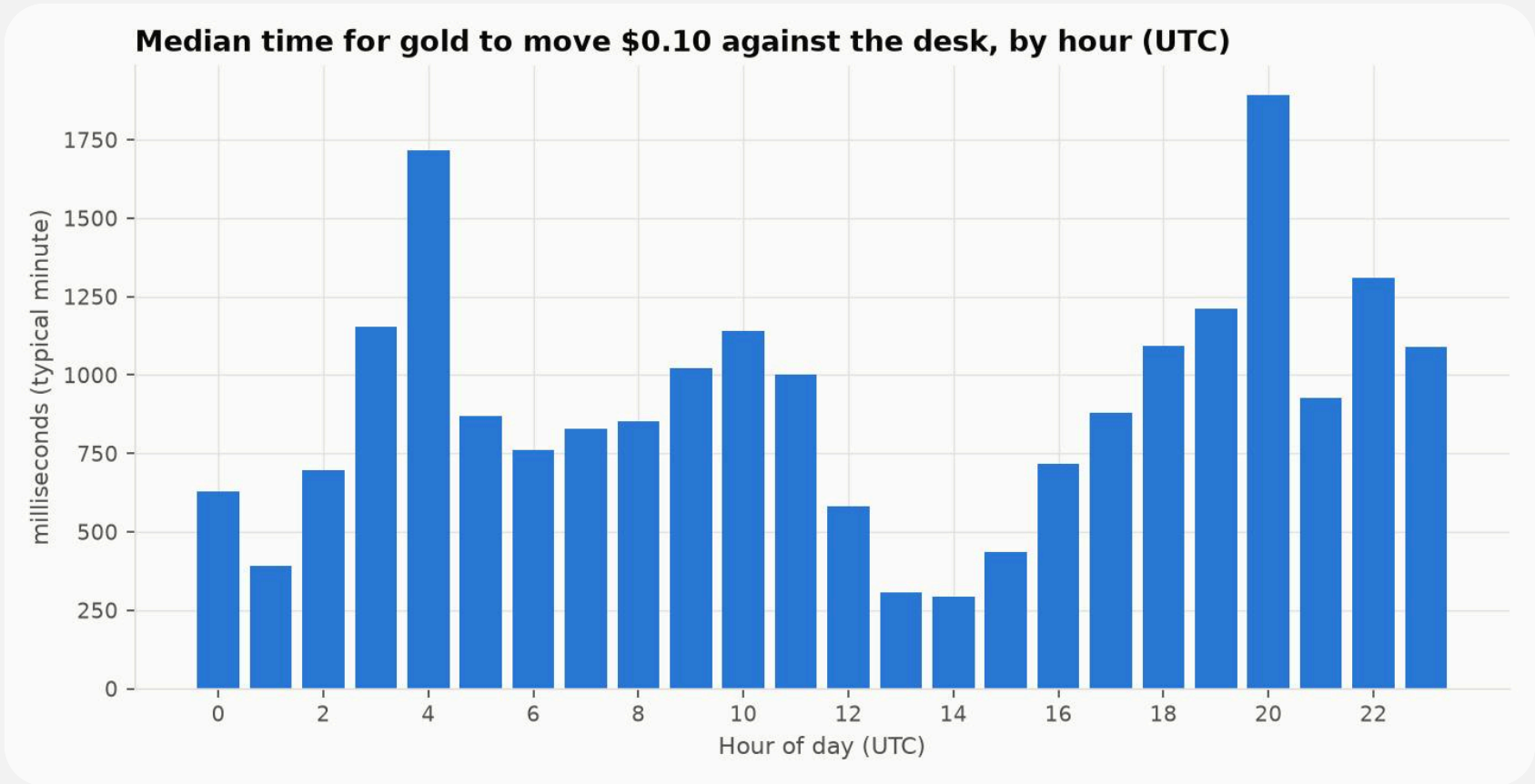
Thank you



Appendix

Backup charts and tables for Q&A

Latency and Throughput



Latency how fast gold moves

- **Median move time:** 0.6–0.85 s, about 0.3 s in the 13–14 UTC London/New York overlap.
- **Typical minute:** a \$2.15 range, about 21× the 10c threshold.
- **Latency budget:** ≤5% of trades fail the check at ~50–90 ms; ≤10% at ~100–170 ms; ≤25% at ~0.4–0.6 s.

Throughput 84,456 trades, Feb–Sep

- **Peak:** 5.9 trades/min (14 Aug, 09:30–10:00).
- **Bursts:** busiest minute 19 trades; busiest second 5, from 5 accounts. Median gap between trades is 24 s.
- **Build for:** bursts of 5–10 simultaneous trades. Only 2% of trades close within 5 s.

Latency Budget vs Measured

What the data says 10c adverse-price check

- **Under ~100 ms:** the check only works if fill → desk order is this fast. At 1 s, ~30% of trades already fail.
- **Strict budget:** ≤1% failing needs ~15–30 ms.
- **Calm vs volatile:** 5% failing allows ~0.7 s in a calm minute, ~3 ms in the most volatile 1% (news spikes).
- **Stricter reading:** if any move to +10c before the order counts, limits roughly halve.
- **Data limit:** nothing finer than 1 s, so sub-second is scaled. The real market is slower there, so these limits are cautious.

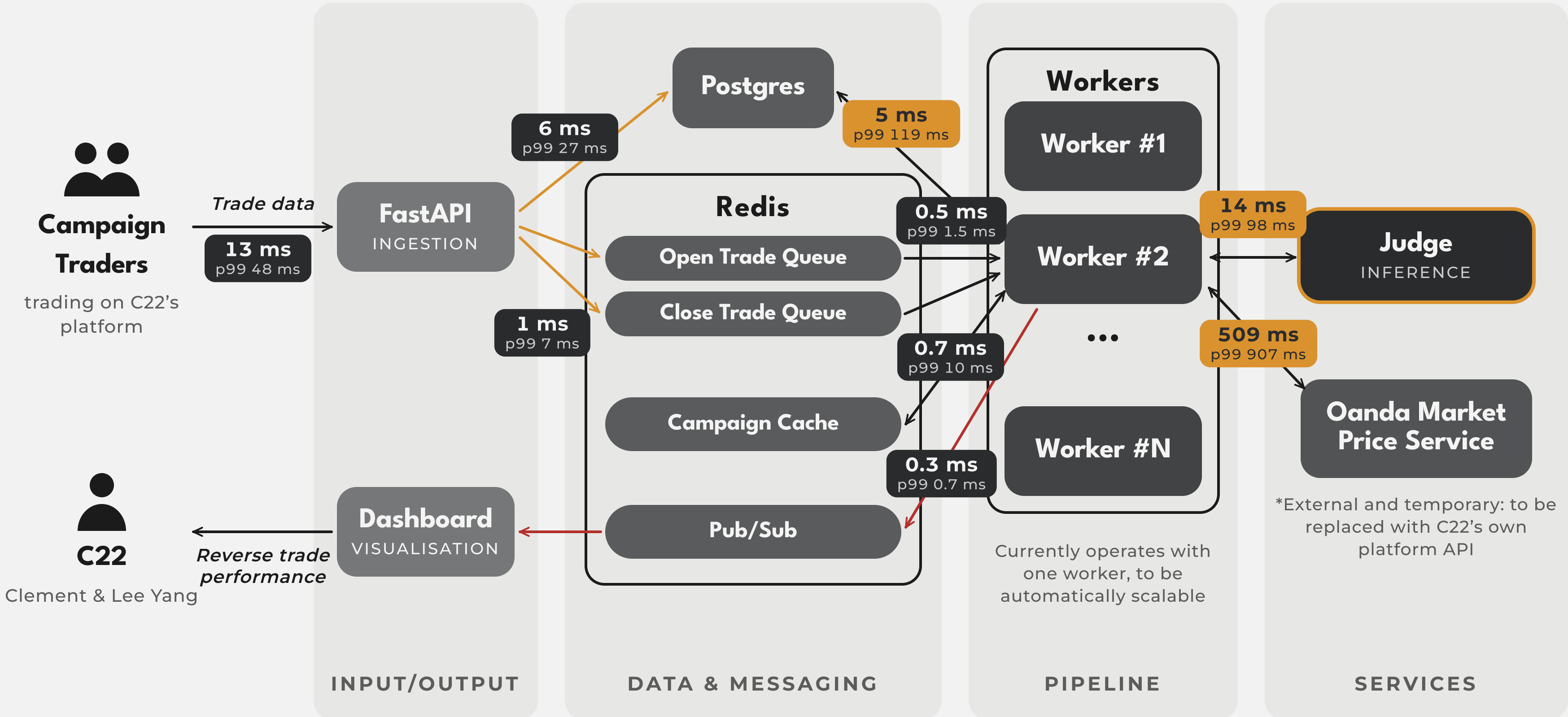
Our peak run

Milliseconds from API receipt. 102 trades: 99 completed, 3 slipped. About 10% pass 100 ms; the desk order waits on OANDA.

	p50	p90	p99	Max
Decision	31	88	228	241
OANDA read	558	746	951	1,850

Where the Time Goes

p50 of 102 open trades, p99 beneath. The clock starts when the API receives the trade.



The Slow 10%

What slows the tail p50 → p99

- **Judge call:** 10 → 87 ms. A new HTTP client is built every call (80 ms at p99). Share one.
- **Counting closed trades:** 2 → 107 ms. One DB query with a bad tail. Check with EXPLAIN, likely a missing index.
- **Storing the open trade:** 8 → 31 ms. Two commits where one would do.

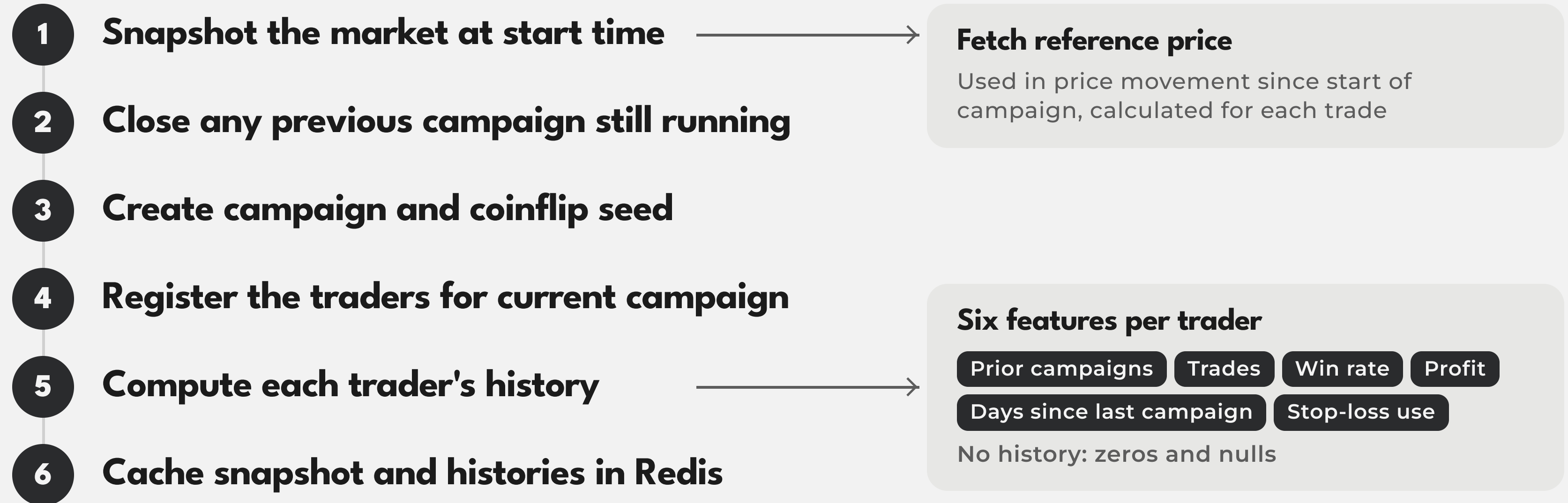
Not the problem ruled out

- **Queue wait:** 0.5 ms (p99 1.5 ms). With 15 workers, one is always free.
- **Redis steps:** under 1 ms at the median.
- **Judge model:** about 3 ms.
- **Bursts of 5:** no slowdown.

**The real problem
is OANDA**

If the desk order
waits for the ~558
ms price read, about
100% of trades miss
the budget.

Workflow 1: Campaign Start



Steps 1–6 run in one database transaction. If any step fails, nothing is saved and a 500 response is sent with failed step

Workflow 2: Open Trade

1 Trade arrives

Trader opens a new position

2 Store trade in Postgres, queue it

Picked up by workers running the open-trade pipeline

ASYNC · OPEN TRADE PIPELINE RUN BY WORKER

3 Read cached trader history from Redis

4 Build 18 features, call the Judge

Inference returns a decision from the features

5 Fetch market price, check slippage

6 Record the simulated reverse position

Market price + Judge decision, stored for reverse profit calculation

Key notes

- Market data will be polled or read from C22 directly, with no response sent.
- Slippage threshold is 0.10 as set by C22, adjusted for this demo because of historical data.

Market price

Currently using the Oanda API as a placeholder; will integrate with C22

Slippage threshold

Max allowed difference in market price between when the trader opened their position and when we open ours

Workflow 3: Close Trade

1 Close trade arrives

Trader closes an existing position

2 Store trade in Postgres, queue it

Picked up by workers to run the close-trade pipeline

ASYNC · CLOSE TRADE PIPELINE RUN BY WORKER

3 Update running stats in Redis

Campaign and trader stats, for subsequent inferences

4 Fetch market price at close time

5 Calculate simulated profit/loss

Based on the Judge's decision

6 Update campaign's live totals

Published to the dashboard

Key notes

- Rolling metrics update in Redis cache even if the system fails to simulate a close position
- Rolling metrics are updated safely with a read-check-write retry pattern, not locks
- Live reverse trading PnL are streamed to the dashboard the moment each trade closes

Workflow 4: Campaign End & Evaluation

1

Campaign end trigger received

Via API call to /end

2

Campaign marked completed in Postgres

3

Any trade still open is force-failed

4

Aggregate values for 3 strategies

On completed trades: Judge, Reverse-all, Coin-flip

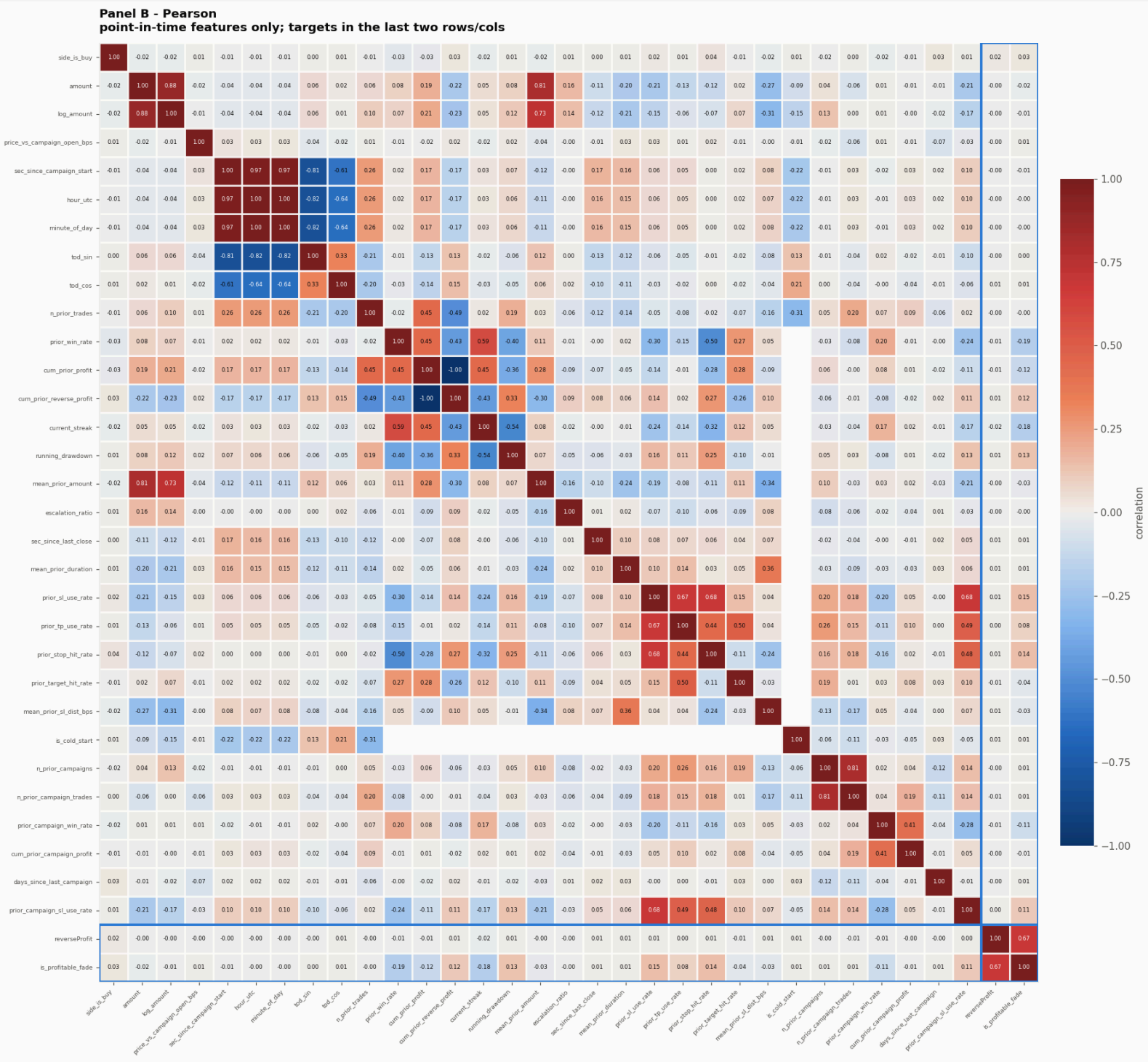
5

Store results, clear campaign cache

Key notes

- **Idempotent closing:** duplicated requests to end an already ended campaign is nullified and does not rerun the flow
- **Final PnL evaluation:** the following strategies are used on the same trade data and compared:
 - Judge decision-led reverse
 - Reverse all trades
 - Coin flip-led reverse

Feature Correlations



Against the target

No feature correlates above 0.19 with whether fading wins

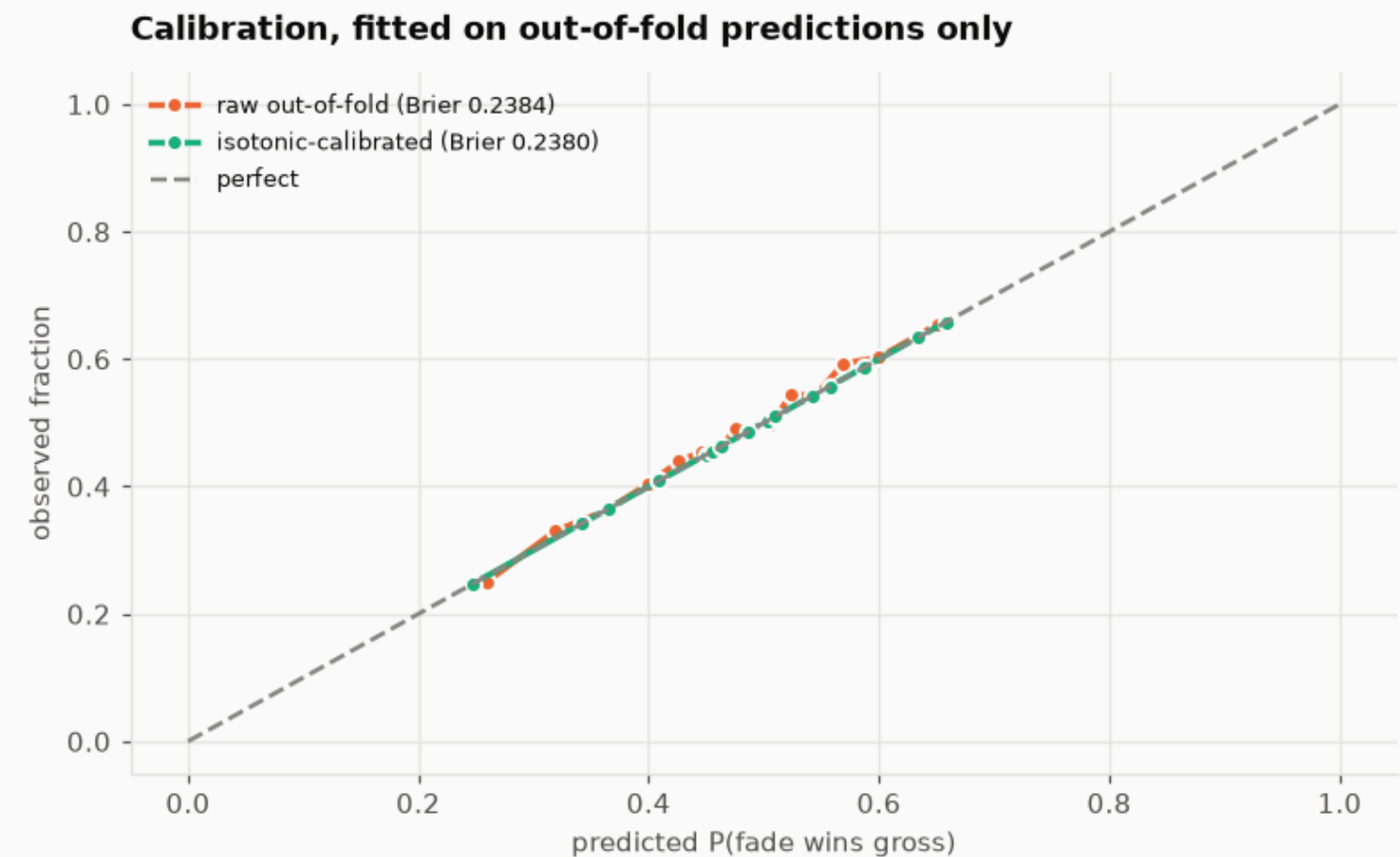
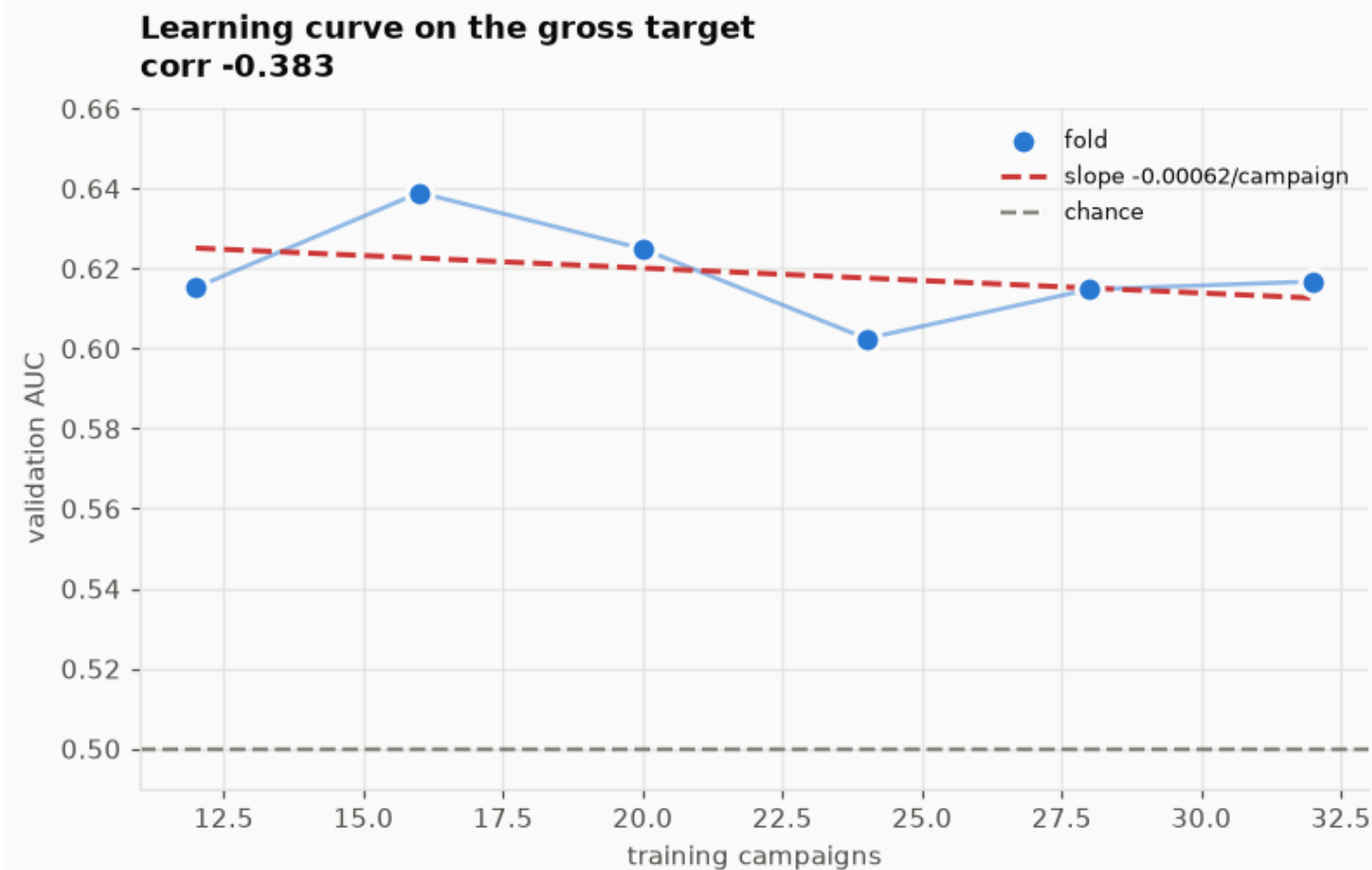
Among themselves

Time-of-day and cumulative-profit features move together, so we read importance with SHAP, not raw correlation

Learning Curve & Calibration

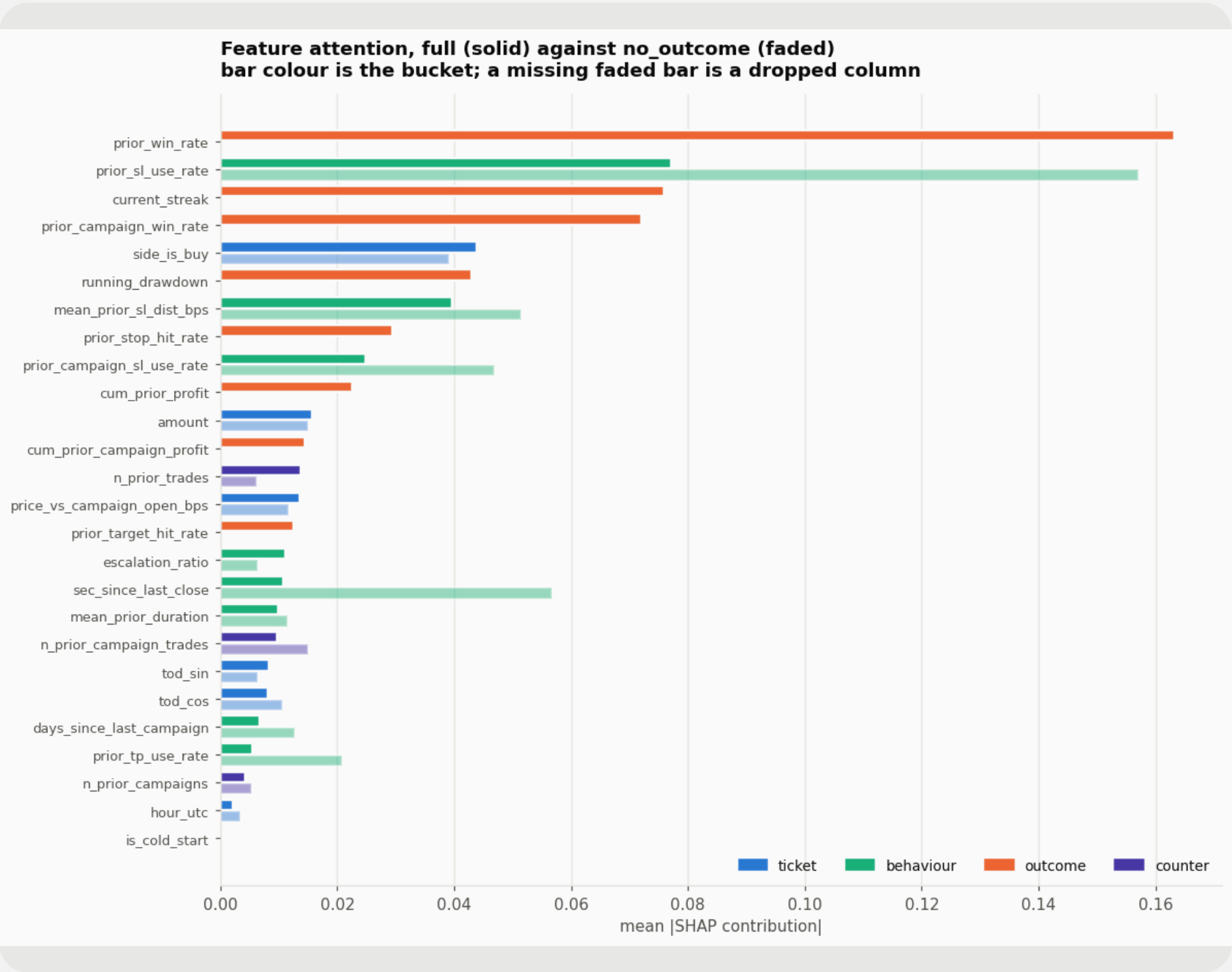
GROSS TARGET · BLOCKED CV

More campaigns do not help yet



Flat learning curve (corr -0.38)
Brier 0.2384 → 0.2380 after calibration

SHAP by Feature



Full model

prior_win_rate alone carries 0.16 mean |SHAP|, then prior_sl_use_rate and current_streak

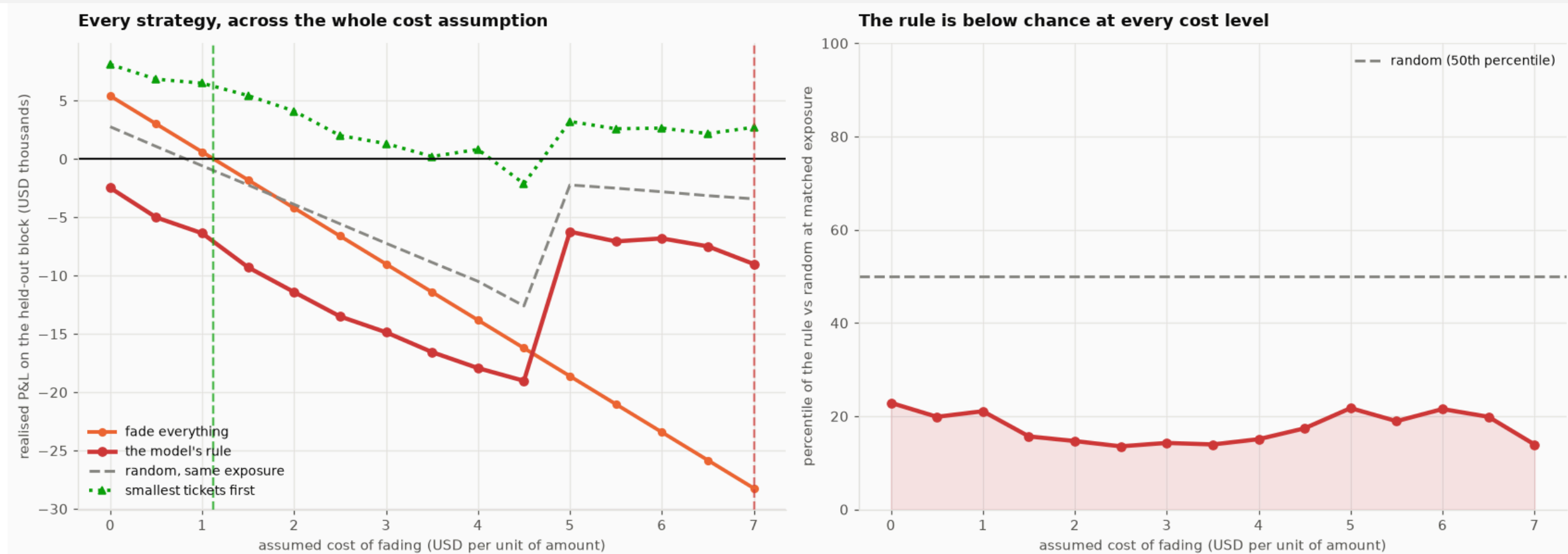
Without outcome history

prior_sl_use_rate takes over, so the model falls back on how traders behave

Cost Assumption Sweep

HELD-OUT CAMPAIGNS 69–77

Every strategy across every cost

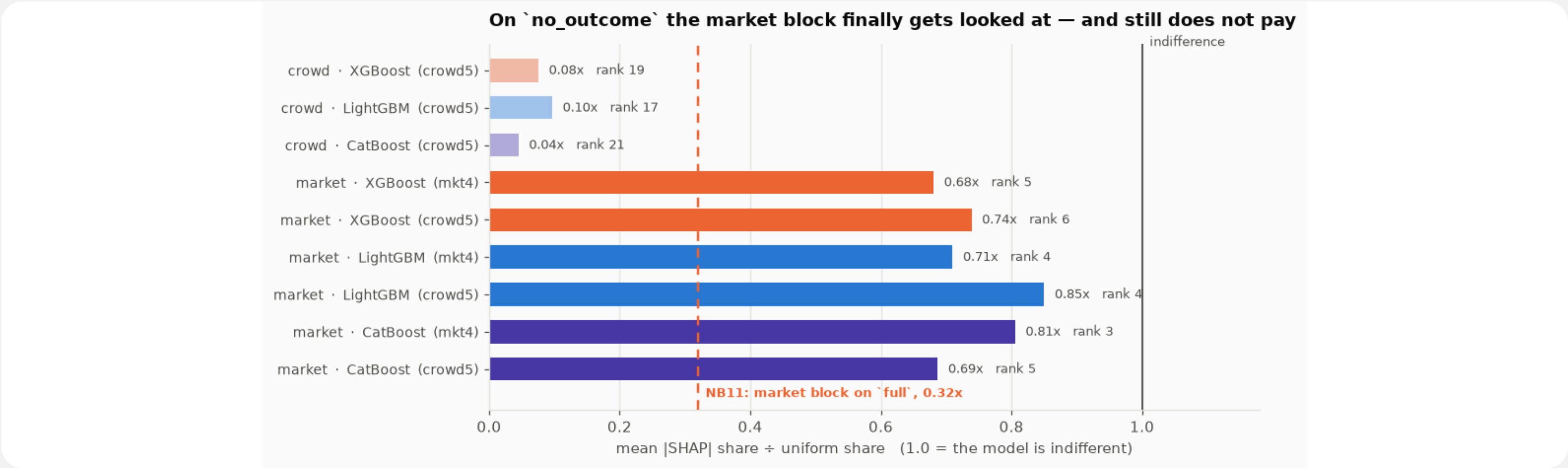


The threshold rule sits at the 13th to 23rd percentile of random at every cost level

Market Data, Revisited

MEAN |SHAP| SHARE ÷ UNIFORM SHARE

Market features get looked at, and still do not pay



AUC neutral to better
Dollars worse in all eight arms

Tried and Dropped

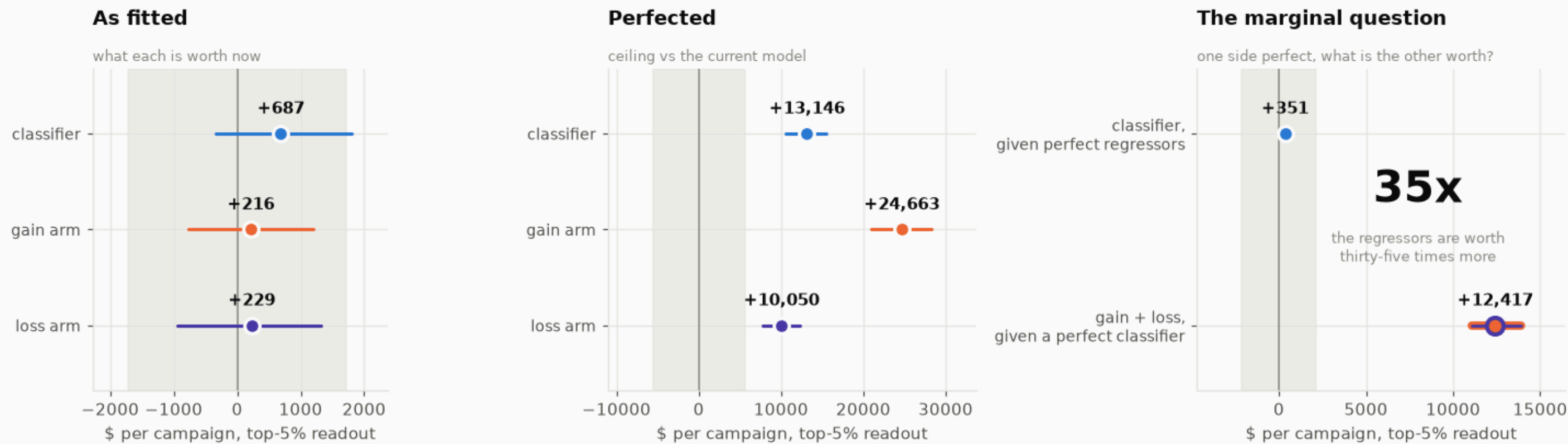
Lever	Result
Market context	Rejected 4×
Stop / target channel	+0.006 AUC at best
Hyperparameter tuning	+0.0028 CV AUC
Tuned GBDT vs logistic	+0.002 AUC
More training campaigns	Flat learning curve
Same-day trade-context block	−\$7,571 at top 5%

Hurdle Decomposition

TOP-5% READOUT · \$ PER CAMPAIGN

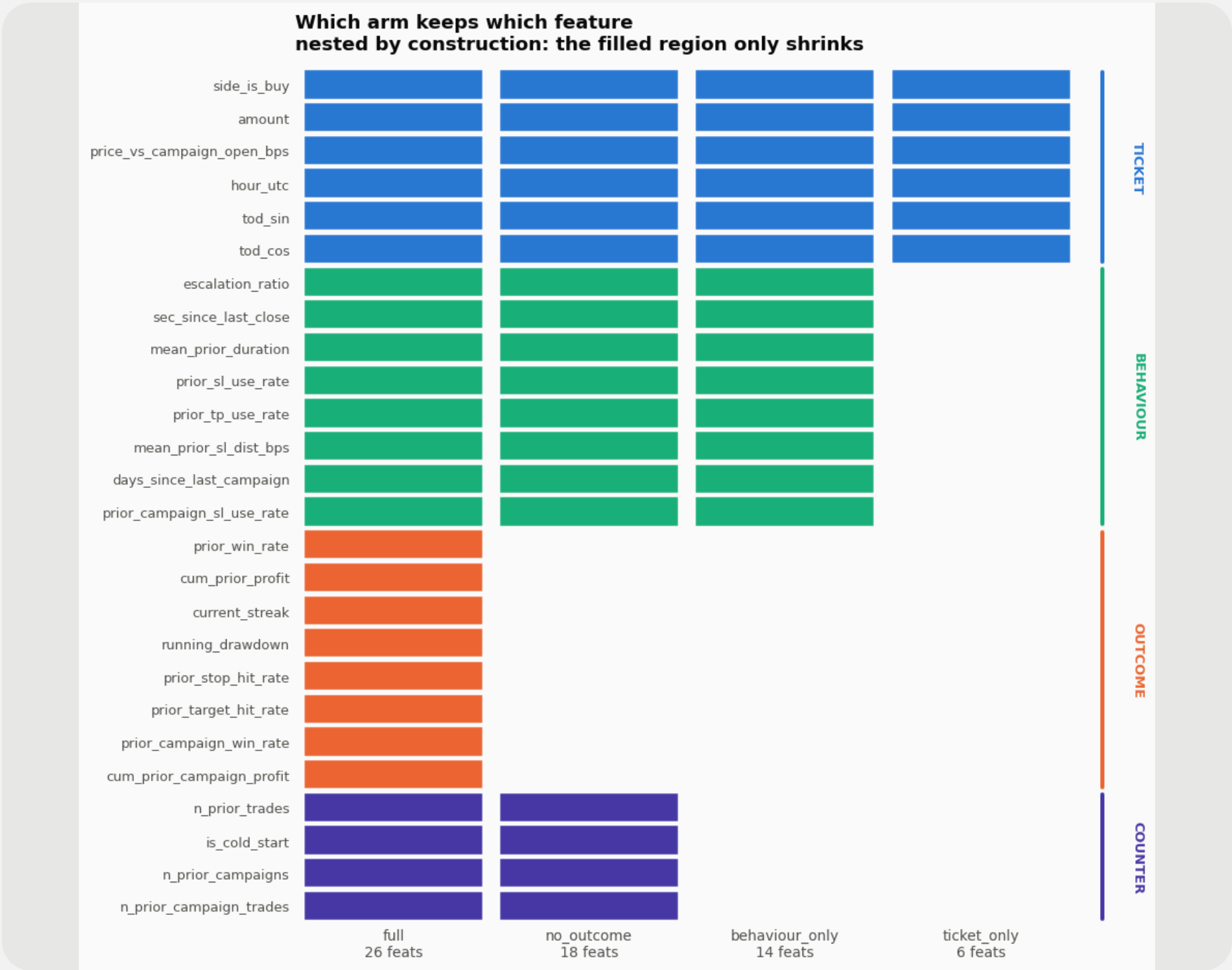
A perfect classifier is worth 1/35th of perfect regressors

Figure 1 · A perfect classifier is worth 1/35th of perfect regressors



+\$351 from a perfect classifier
+\$12,417 from perfect regressors

The 26 Features



Four arms, nested

Full (26) → no outcome (18) →
behaviour only (14) → ticket only (6)

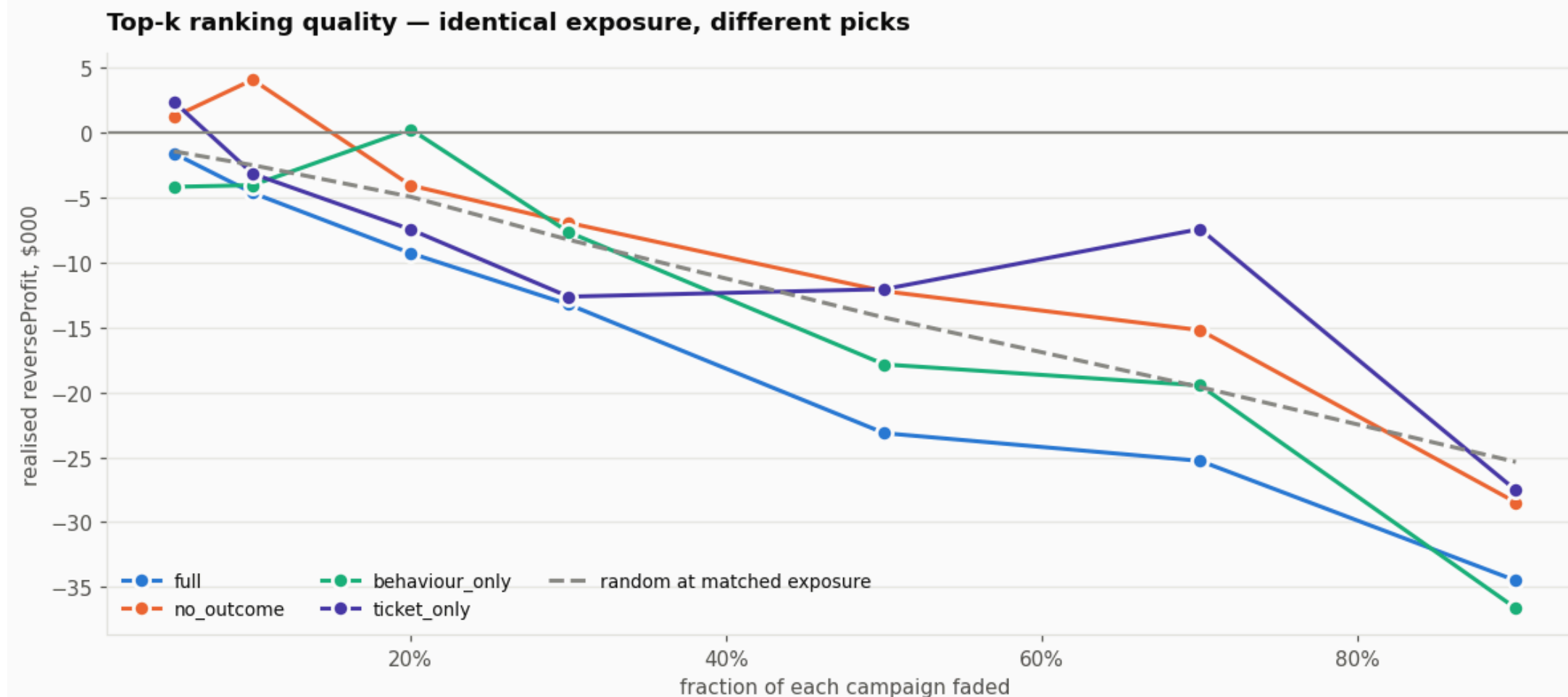
Outcome goes first

Win/loss history is the block we
remove to test behaviour against luck

Does the Ranking Make Money?

HELD-OUT CAMPAIGNS 69-77

Top-k ranking quality, identical exposure



Full model: below random at every cut
No arm clearly beats random

Judge v5 Hyperparameters

Tree shape

Shared by all three models

max_depth	4
num_leaves	15
min_child_samples	47
learning_rate	0.0707

Sampling

Shared; includes L1/L2 penalties

subsample	0.752
colsample_bytree	0.638
reg_lambda	4.125
reg_alpha	2.812

Per model

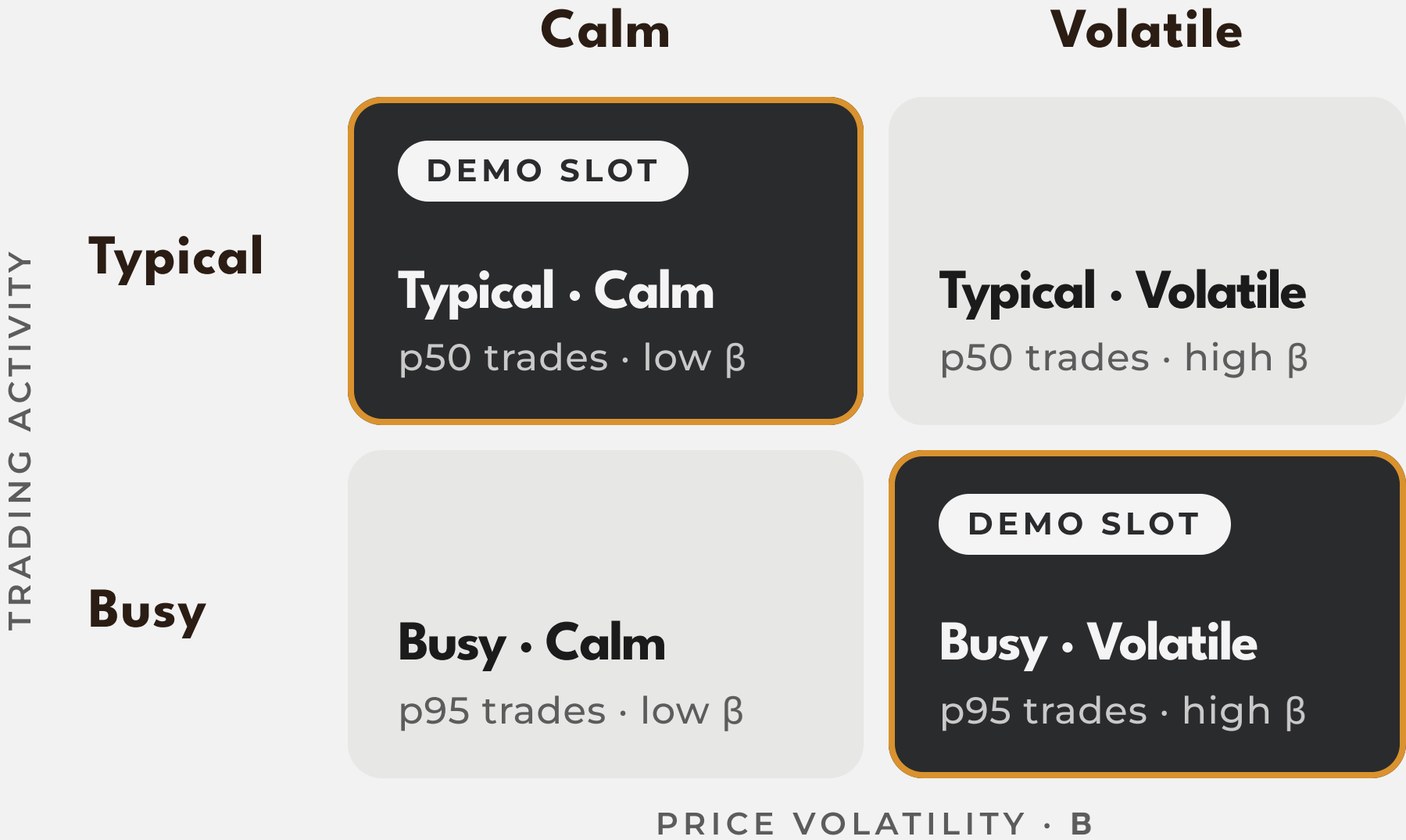
Early stop 100 rounds, refit at median best

	Loss	Trees
Win?	binary	44
Gain	huber	6,392
Loss	huber	6,412

How We Picked the Demo Data

Two lenses

How busy the slot is, and how volatile the market is



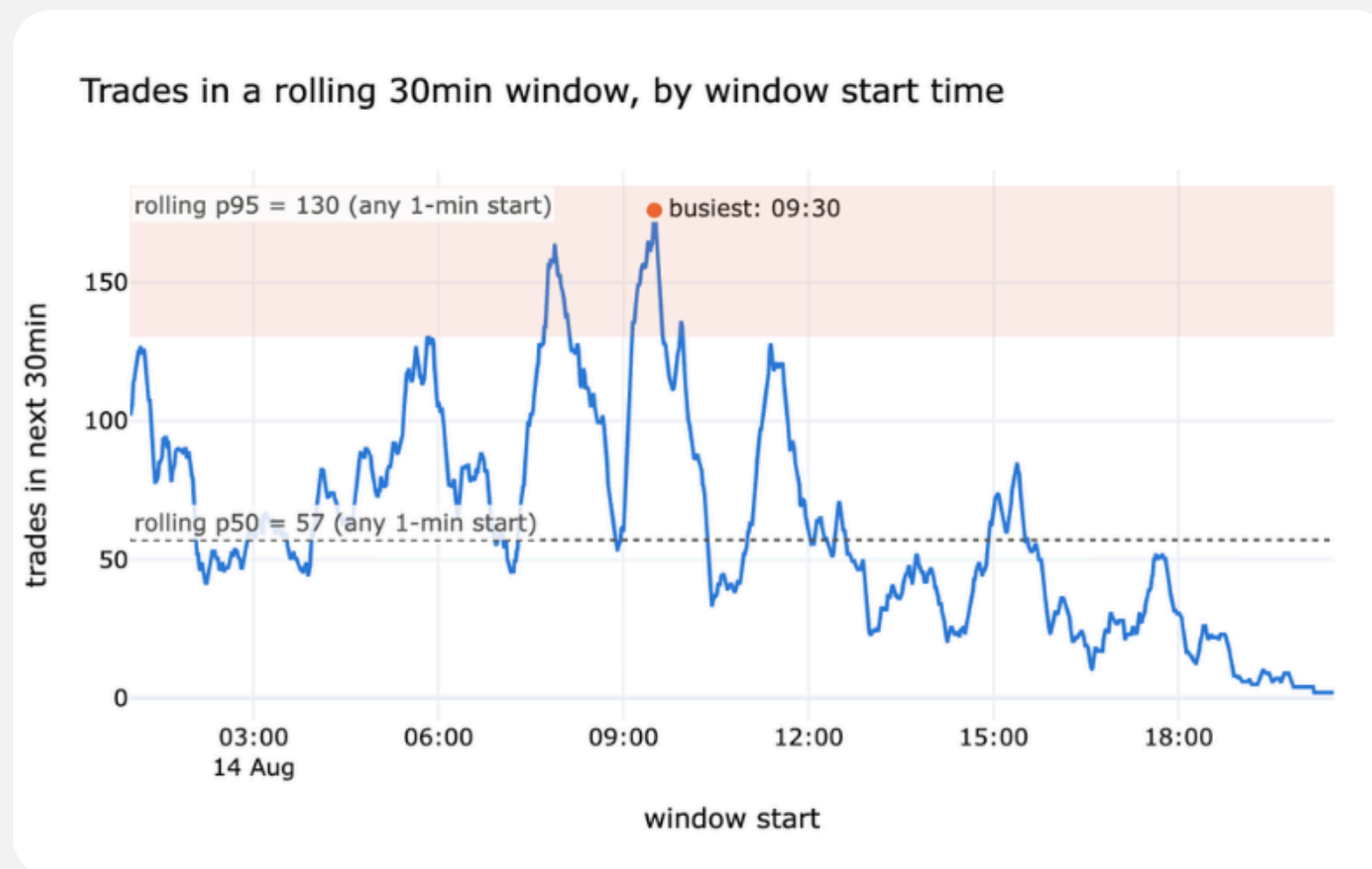
Activity Trades per slot, p50 vs p95

Volatility Beta (β) vs Gold Futures

The Data Behind Our Choice

ACTIVITY

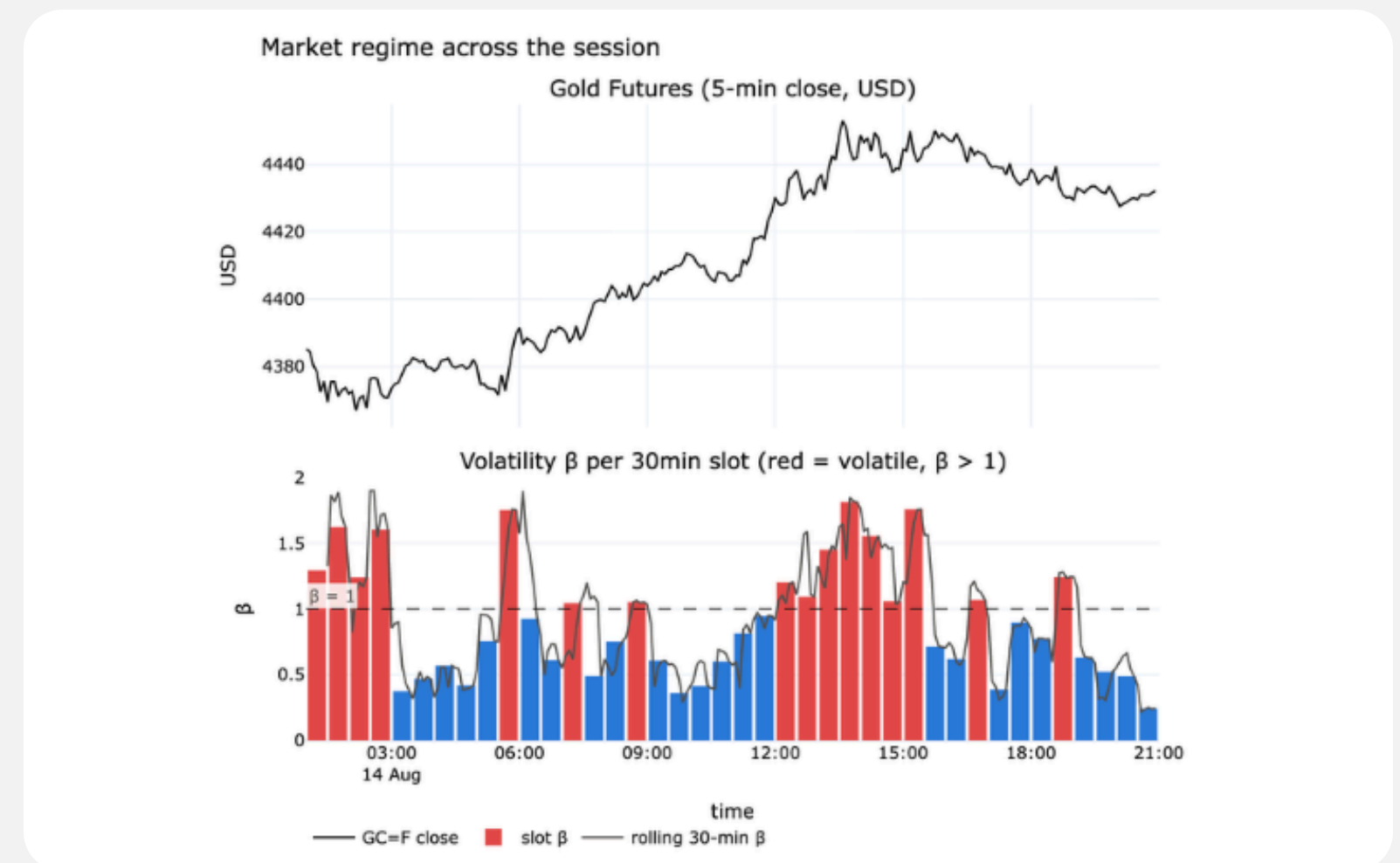
Trades per slot



Typical $\approx$ p50
Busy = p95 and above

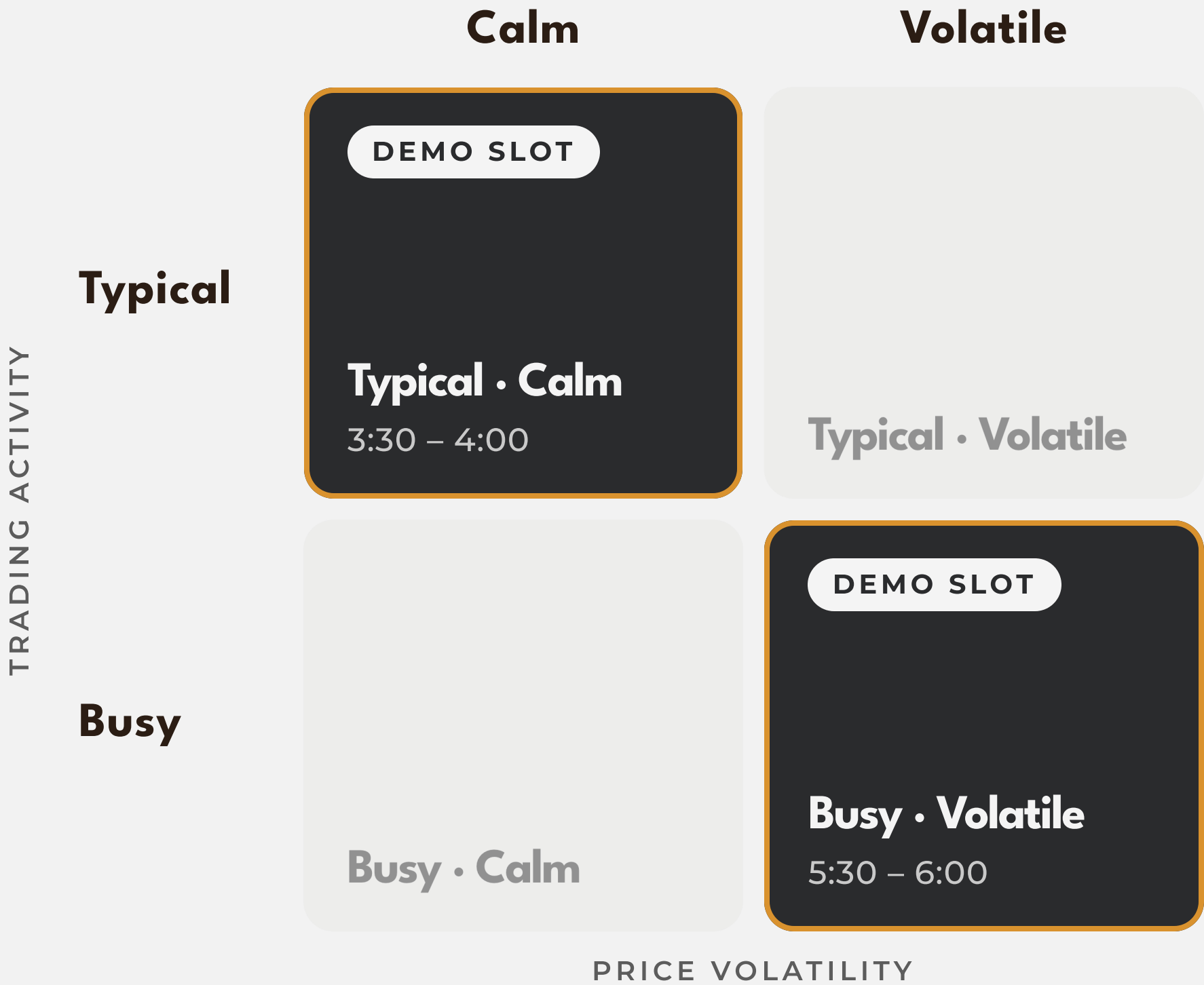
VOLATILITY

Beta vs Gold Futures



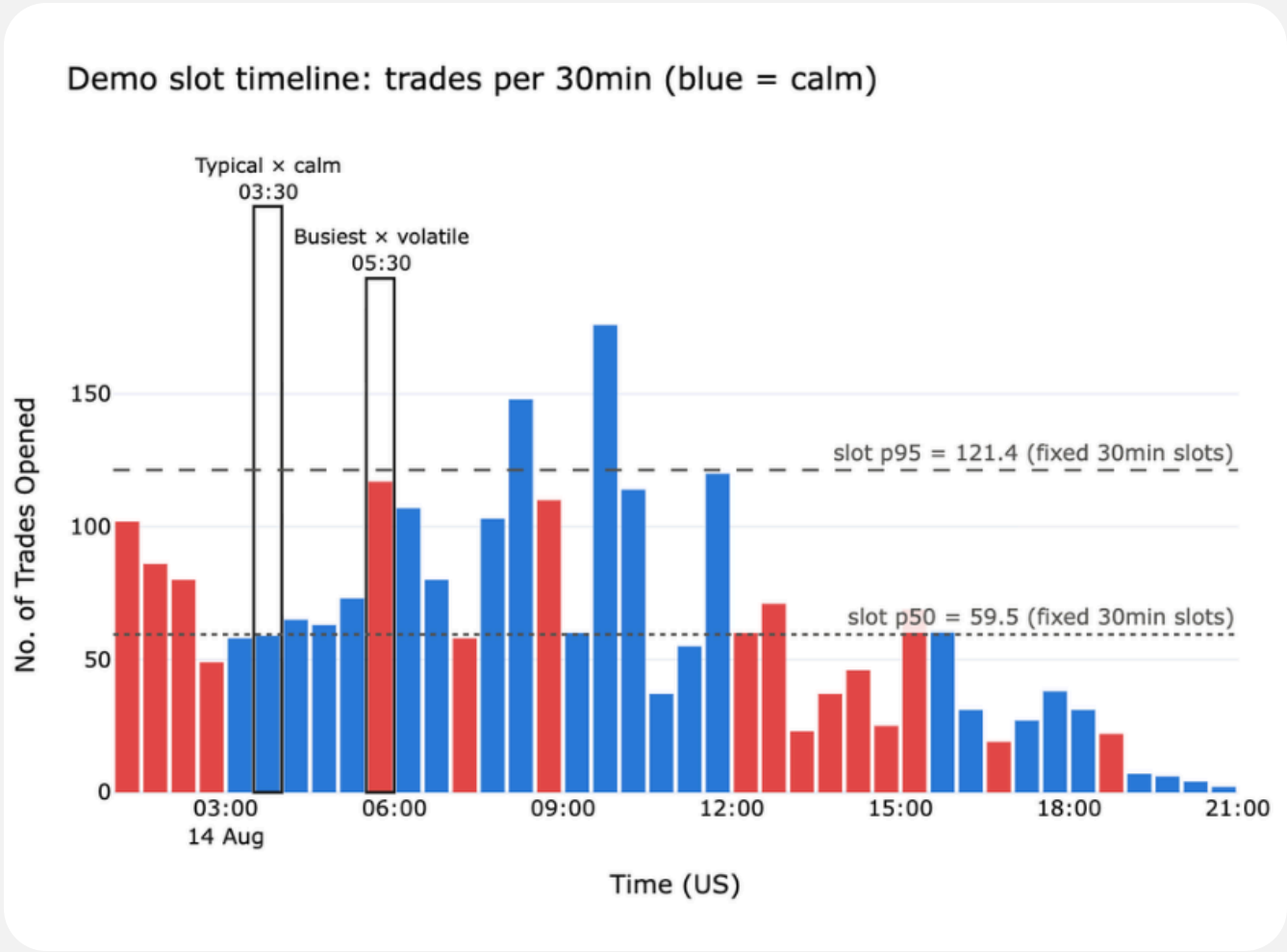
Calm: $\beta \leq 1$
Volatile: $\beta > 1$

Chosen Timeslots



FIXED 30-MIN SLOTS • US TIME, 14 AUG

Demo slot timeline



p95 = 121.4 trades
p50 = 59.5 trades